

SRINIVAS UNIVERSITY

Mukka – 574146, Mangalore, India, Phone: 0824-2477456

**(State Private University Established by Karnataka Govt. Act No. 42 of 2013
empowered to award degrees under Section 22 Of UGC Act of UGC, New Delhi, &
Member of Association of Indian Universities, New Delhi)**

Web: www.srinivasuniversity.edu.in, Email: info@srinivasuniversity.edu.in



OBJECT ORIENTED PROGRAMMING

STUDY MATERIAL

For

**Ist Sem Bachelor of Computer Application
(2021-22)**

INSTITUTE OF COMPUTER SCIENCE & INFORMATION SCIENCES

Paper : 21CAC-4/SD Theory/Week: 3 Hours Credits: 3	OBJECT ORIENTED PROGRAMMING	Hours: 40 IA : 50 Exam: 50
Course Objectives: - To understand concept of Object Oriented Programming - To create programs using C++. - To implement all the concepts of OOP using C++.		
Course Outcomes: After the completion of the course students will be able CO1: Recall the difference between the top-down and bottom-up approach in programming. CO2: Demonstrate the object-oriented programming approach in with C++. CO3: Develop programs using the concepts C++.		
Module – I		8 hrs
Principles of Object Oriented programming: Basic Concepts, benefits and Applications. Classes and objects: Specifying a class, creating objects, accessing class members, defining member functions-inside the class definition and outside the class definition, Arrays within a class. Memory allocation for objects- Static data members, static member functions Arrays of objects, objects as function arguments, returning objects, Friend functions Teaching Methodology: Chalk and talk method: Classes and objects: Specifying a class, creating objects, accessing class members, defining member functions-inside the class definition and outside the class definition, Arrays within a class. PowerPoint presentation: Memory allocation for objects- Static data members, static member functions Arrays of objects, objects as function arguments, returning objects, Friend functions Self-study material: Principles of Object Oriented programming: Basic Concepts, benefits and Applications.		
Module – II		8 hrs
Constructors and destructors: Parameterized constructors, constructors with default arguments, overloading constructors, copy constructor, destructors. Operator overloading and Type Conversions: Defining, overloading unary operators, Overloading binary operators, Manipulation of strings using operator overloading, rules for overloading operators. Teaching Methodology: Chalk and talk method: Constructors and destructors: Parameterized constructors, constructors with default arguments, overloading constructors, copy constructor, destructors. Power point presentation: Operator overloading and Type Conversions: Defining Video: Overloading binary operators, Manipulation of strings using operator overloading, rules for overloading operators. Self-study material: overloading unary operators		
Module – III		8 hrs

Function Overloading using class: Definition of Function, Ways to overload a function By changing number of Arguments, By having different types of argument. Inheritance: Defining a derived class, single inheritance, protected members, multilevel inheritance, multiple inheritance, hierarchical inheritance, hybrid inheritance, virtual base classes, abstract classes, constructors in derived classes..

Teaching Methodology:

Chalk and talk method: Function Overloading using class: Definition of Function, Ways to overload a function By changing number of Arguments, By having different types of argument. PowerPoint presentation: Inheritance: Defining a derived class, multiple inheritance, hierarchical inheritance, hybrid inheritance, virtual base classes, abstract classes, constructors in derived classes..

Self-study material: single inheritance, protected members, multilevel inheritance

Module – IV**8 hrs**

Pointers: Introduction to Pointers, Pointers to Objects, this pointer, Pointers to derived classes. Virtual Functions and Polymorphism: Function Overriding, Virtual functions, Abstract Class and pure virtual functions.

Teaching Methodology:

Chalk and talk method: Function Overriding, Virtual functions, Abstract Class and pure virtual functions.

Power Point presentation: Pointers: Introduction to Pointers, Pointers to Objects, this pointer, Pointers to derived classes. Virtual Functions and Polymorphism

Module – V**8 hrs**

Templates: Class templates, class templates with multiple parameters, function templates, function templates with multiple parameters, overloading of Template functions and member function templates. Exception Handling: Basics, Mechanism, throwing mechanism, catching mechanism, Re-throwing an Exception and specifying exceptions.

Teaching Methodology:

Chalk and talk method: overloading of Template functions and member function templates.

PowerPoint presentation: Templates: Class templates, class templates with multiple parameters, function templates, function templates with multiple parameters.

Hands-on exercise in Laboratory: Exception Handling: Basics, Mechanism, throwing mechanism, catching mechanism, Re-throwing an Exception and specifying exceptions.

Text Books

1	E Balagurusamy, Object Oriented Programming with C++, 5th Edition, Tata McGraw Hill Publication.
2	D Ravichandran, Programming with C++, Third Edition, McGraw hill 2011

Reference Book

1	Robert Lafore, Oriented Programming in C++, Galgotia Publications Pvt. Ltd, 2006.
2	Herbert schildt, C++ A Beginner's Guide, McGraw Hill, 2003
3	Bjarne Stroustrup, The C++ Programming Language, Addison-Wesley Professional, 2013

OBJECT ORIENTED PROGRAMMING

Total Hours: 40**Module 1: Principles of Object Oriented programming****Session1:** Basic Concepts, benefits and Applications.**Session2:** Classes and objects: Specifying a class, creating objects**Session3:** accessing class members, defining member functions-inside the class definition and outside the class definition**Session4:** Memory allocation for objects- Static data members**Session5:** static member functions Arrays of objects**Session6:** objects as function arguments**Session7:** returning objects**Session8:** Friend functions**Module 2: Constructors and destructors****Session9:** Parameterized constructors**Session10:** constructors with default arguments**Session11:** overloading constructors**Session12:** copy constructor, destructors**Session13:** Operator overloading and Type Conversions: Defining**Session14:** overloading unary operators, Overloading binary operators**Session15:** Manipulation of strings using operator overloading**Session16:** rules for overloading operators**Module 3: Function Overloading using class****Session17:** Definition of Function, Ways to overload a function By changing number of Arguments**Session18:** Ways to overload a function By having different types of argument**Session19:** Inheritance: Defining a derived class, single inheritance**Session20:** protected members**Session21:** multilevel inheritance, multiple inheritance**Session22:** hierarchical inheritance, hybrid inheritance**Session23:** virtual base classes, abstract classes**Session24:** constructors in derived classes**Module 4: Pointers****Session25:** Pointers to Objects**Session26:** this pointer**Session27:** Pointers to derived classes**Session28:** Virtual Functions and Polymorphism: Function Overriding**Session29:** Virtual functions**Session30:** Abstract Class**Session31:** Example Programs**Session32:** pure virtual functions

Module 5: Templates

Session33: Class templates, class templates with multiple parameters

Session34: function templates, function templates with multiple parameters

Session35: overloading of Template functions and member function templates.

Session36: Exception Handling: Basics, Mechanism

Session37: throwing mechanism

Session38: catching mechanism

Session39: Re-throwing an Exception

Session40: specifying exceptions

Scheme of Evaluation:

The paper carries 100 marks out of which 50 marks will be allotted to external examination and 50 marks will be allotted to the internal assessment.

Internal assessment marks will be calculated as follows

1. Performance in 2 IA examinations will be converted out of 20 marks
2. Attendance 10 marks
3. Assignment 10 marks.
4. Viva 10 Marks

Total 50 marks.

External examination marks will be as follows

1. 1 marks questions 10 out of 12 1 X 10 = 10 marks.
2. One full question out of 2 full questions in each unit carries 8 X 5 = 40 marks

Total 50 marks.

In order to clear this paper minimum 50% marks must be scored both in internal and well as external examination.

UNIT-I
CHAPTER-1
PRINCIPLES OF OBJECT ORIENTED PROGRAMMING

INTRODUCTION

Since the invention of the computer, many programming approaches have been tried. These include techniques such as modular programming, top-down programming, bottom-up programming and structured programming. The primary motivation in each has been the concern to handle the increasing complexity of programs that are reliable and maintainable. These techniques have become popular among programmers over the last two decades.

With the advent of languages such as C, structured programming became very popular and was the main technique of the 1980s. Structured programming was a powerful tool that enabled programmers to write moderately complex programs fairly easily. However as the programs grew larger, even the structured approach failed to show the desired results in terms of bug-free, easy-to-maintain, and reusable programs.

Object-Oriented Programming (OOP) is an approach to program organization and development that attempts to eliminate some of the pitfalls of conventional programming methods by incorporating the best of structured programming features with several powerful new concepts. It is a new way of organizing and developing programs and has nothing to do with any particular language. However, not all languages are suitable to implement the OOP concepts easily.

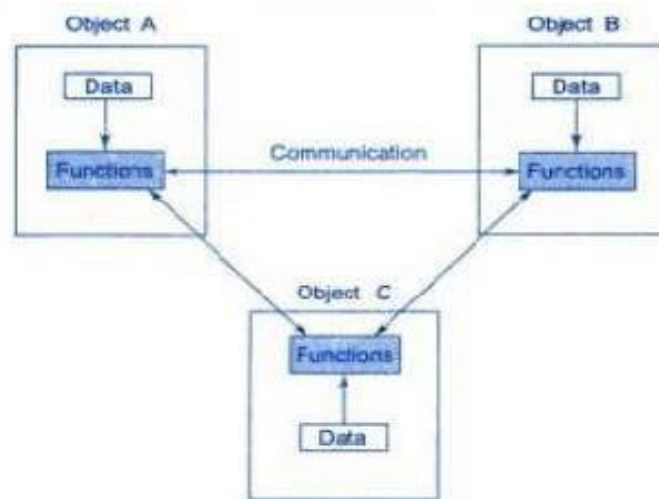
Object oriented programming is the principle of design and development of programs using modular approach. Object oriented programming approach provides advantages in creation and development of software for real life applications. The advantage is that small modules of program can be developed in shorter span of time and these modules can be shared by a number of applications. The basic element of object oriented programming is the data. The programs are built by combining data and functions that operate on the data. In this chapter we learn about the basic concepts, advantages and terminologies used in Object oriented programming. Some of the object oriented programming languages are C++, Java, C# and so on.

OBJECT ORIENTED PROGRAMMING

The object oriented programming methods use data as the main element in the program. The data is tied to the function that operates on the data and the other functions cannot modify the data tied to a given function. Thus in object oriented programming, a problem is decomposed into a number of components called objects. **An object is a collection of set of data known as member data and the functions that operate on these data known as**

member functions or Methods. The member data are encapsulated in an object and then can be accessed or modified only by the member functions. An object can be accessed only if permitted by other member functions. Various objects of a program can interact with each other by sending messages.

Object oriented programming methods modularize a program by creating memory area for data and member functions (methods) together as a single entity. All objects are created according to the specifications of the entity defined. Object is the basic unit of OOP. To design OO Model, first a set of classes are defined. A class is a Template from which objects are created. The Template of a class specifies the data, member functions and their attributes.



Some of the features of object-oriented programming are:

- Emphasis is on data rather than procedure.
- Programs are divided into what are known as objects.
- Data structures are designed such that they characterize the objects.
- Functions that operate on the data of an object are tied together in the data structure.
- Data is hidden and cannot be accessed by external functions.
- Objects may communicate with each other through functions.
- New data and functions can be easily added whenever necessary.
- Follows bottom-up approach in program design.

BASIC CONCEPTS OF OBJECT ORIENTED PROGRAMMING

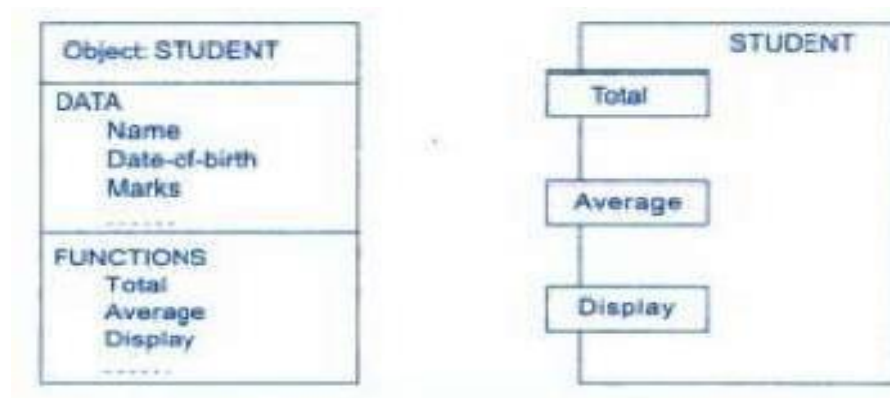
The following are the major concepts of any object oriented programming language:

- Objects
- Classes
- Data abstraction
- Data encapsulation

- Inheritance
- Polymorphism
- Dynamic binding
- Message passing

Objects

Objects are the basic run-time entities in an object oriented system. Objects are basic building blocks for designing programs. An object may represent a person, place, a bank account, a table of data or any item that the program has to handle. An object is a collection of data members and associated member functions. Each object is identified by a unique name. Every object must be a member of a particular class. Ex: Apple, orange, mango are the objects of class fruit. Objects take up space in memory and have address associated with them. For example, structure variable in a C program. At the time of execution of a program, the objects interact by sending messages to one another. The objects can interact with one another without having to know the details of data or functions within an object.



Classes

The objects can contain data and code to manipulate the data. The entire set of data and code of an object can be made a user defined data type with the help of a class. Therefore objects are variables of the type class. **Class is a template/blue-print representing a group of objects that share common properties and relationships.** Classes are user-defined data types and behave like the built-in types of a programming language. A class is a way of grouping objects having similar characteristics. Once a class is defined, any number of objects of that class is created. For example, planets, sun, moon are members of class solar system. Classes are user defined data types. A class can hold both data and functions.

Data abstraction

Data abstraction permits the user to use an object without knowing its internal working. **Abstraction refers to the process of representing essential features without including background details or explanations.** Classes use the concept of abstraction and are defined

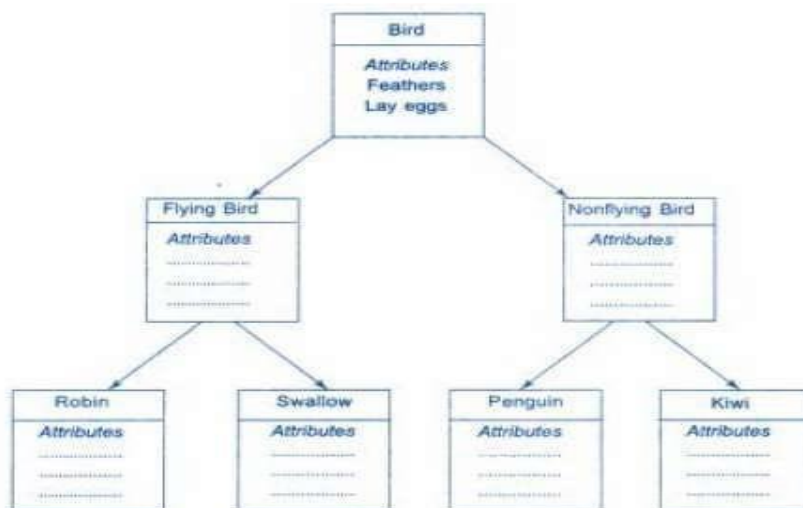
as a list of abstract attributes such as size, weight and cost, and functions to operate on these attributes. They encapsulate all the essential properties of the objects that are to be created. **The attributes are called as data members because they hold information. The functions that operate on these data are called methods or member functions.** Since the classes use the concept of data abstraction, they are known as **AbstractData Types (ADT)**.

Data encapsulation

The wrapping up of data and functions into a single unit is known as encapsulation. Data encapsulation combines data and functions into a single unit called class. Data encapsulation will prevent direct access to data. The data can be accessed only through methods (function) present inside the class. The data cannot be modified by an external non-member function of a class. Data encapsulation enables data hiding or information hiding.

Inheritance

Inheritance is the process by which objects of one class acquire the properties of objects of another class. In OOP, the concept of inheritance provides the idea of reusability. This means that we can add additional features to an existing class without modifying it. Thus the process of forming a new class from an existing class is known as Inheritance. **The existing class is known as base class. The new class is known as derived class.** The derived class shares some of the properties of the base class. Therefore a code from a base class can be reused by a derived class. In addition to this the new class may combine features from two different base classes too.

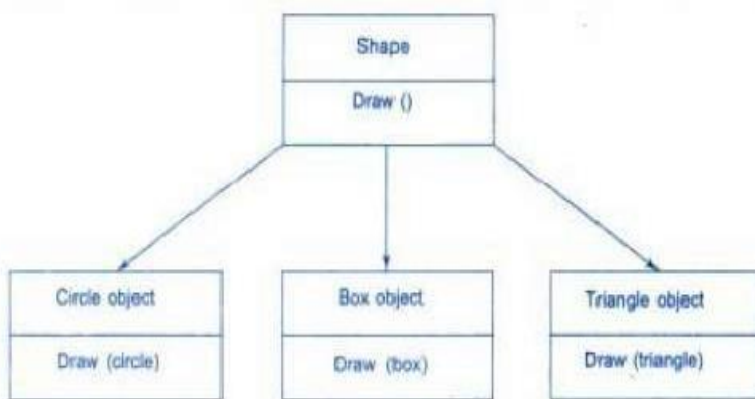


Polymorphism

Polymorphism is another important feature of object oriented programming where a function can take multiple forms based on the type of arguments, number of arguments and data type of return value. The ability of an operator and function to take multiple forms is known as polymorphism. Consider the addition operation. In addition of

two numbers the result is the sum of two numbers. In addition of two strings the operation is string concatenation. When an operator behaves differently based on operands, then it is said that operator is overloaded. Overloading allows objects to have different meaning depending upon context. **There are two types of overloading namely 1.Operator overloading, 2.Function overloading. The process of making an operator to exhibit different behaviours in different instances is known as operator overloading.**

Similarly when same function is used for multiple tasks in the same program by changing argument type and number, it is known as function overloading. Below figure illustrates that a single function name can be used to handle different numbers and different types of arguments.



Dynamic Binding

Binding refers to the linking of a procedure call to the code to be executed in response to the call. **Dynamic binding (also known as late binding) means that the code associated with a given procedure call is not known until the time of call at run-time.** It is associated with polymorphism and inheritance.

Message Passing

An object-oriented program consists of a set of objects that communicate with each other. The process of programming in an object-oriented language, therefore involves following steps:

1. Creating classes that define objects and their behaviour.
2. Creating objects from class definition.
3. Establishing communication among objects.

Objects communicate with one another by sending and receiving information much the same way as people pass messages to one another. **A message for an object is request for execution of procedure. The request will involve a procedure (function) in the receiving object that generates desired results. Message passing involves specifying the name of object, the name of the function (message) and the information to be sent.**

ADVANTAGES OR BENEFITS OF OOP

OOP offers several benefits to both the program designer and the user. Object-orientation contributes to the solution of many problems associated with the development and quality of software products. The new technology promises greater programmer productivity, better quality of software and lesser maintenance cost.

The principal advantages are:

- Through inheritance, we can eliminate redundant code and extend the use of existing classes.
- We can build programs from the standard working modules that communicate with one another, rather than having to start writing the code from scratch. This leads to saving of development time and higher productivity.
- The principle of data hiding helps the programmer to build & secure programs that cannot be invaded by code in other parts of the program.
- It is possible to have multiple instances of an object to co-exist without any interference.
- It is possible to map objects in the problem domain to those in the program.
- It is easy to partition the work in a project based on objects.
- The data-centered design approach enables us to capture more details of a model in implementable form.
- Object-oriented systems can be easily upgraded from small to large systems.
- A message passing techniques for communication between objects makes the interfacedescriptions with external systems much simpler.
- Software complexity can be easily managed.

While it is possible to incorporate all these features in an object-oriented system, their importance depends on the type of the project and the preference of the programmer. There are a number of issues that need to be tackled to reap some of the benefits stated above. For instance, object libraries must be available for reuse. The technology is still developing and current products may be superseded quickly. Strict controls and protocols need to be developed if reuse is not to be compromised.

OBJECT-ORIENTED LANGUAGES

Object-oriented programming is not the right of any particular language. Like structured programming, OOP concepts can be implemented using languages such as C and Pascal. However, programming becomes clumsy and may generate confusion when the programs grow large. A language that is specially designed to support the OOP concepts makes it easier to implement them.

The languages should support several of the OOP concepts to claim that they are object oriented. Depending upon the features they support, they can be classified into the following **two categories**:

1. Object-based programming languages, and
2. Object-oriented programming languages.

Object-based programming is the style of programming that primarily supports encapsulation and object identity. Major features that are required for object-based programming are:

- Data encapsulation
- Data hiding and access mechanisms
- Automatic initialization and clear-up of objects
- Operator overloading

Languages that support programming with objects are said to be object-based programming languages. They do not support inheritance and dynamic binding. Ada is a typical object based programming language.

Object-oriented programming incorporates all of object-based programming features along with two additional features, namely, inheritance and dynamic binding. Object-oriented programming can therefore be characterized by the following statement:

LIMITATIONS OF OOP

The main disadvantages of using Object oriented programming are:

- OOP software is not having set standards.
- The adaptability of flow diagrams and object oriented programming using classes and objects is a complex process.
- To convert a real world problem into an object oriented model is difficult.
- The classes are overly generalized.

APPLICATIONS OF OBJECT ORIENTED PROGRAMMING

Object oriented programming approach is an easier method to design and implement programs. The programs are easier to upgrade and modify. The standard class libraries can be used by the programmers so that development time is minimized. The graphical user interface design for windows operating system using object oriented programming is the most interesting feature of programming.

The common application areas of Object Oriented programming are:

- Computer Graphic applications.
- CIM/CAD/CAM software.
- Object –oriented Databases.
- User Interface design such as windows.
- Real-time systems.
- Simulation and Modelling.
- Artificial intelligence and expert systems.
- Hypertext, Hypermedia and Expertext.
- Neural Networks and Parallel Programming.
- Decision Support and Office Automation Systems.

CHAPTER-2

CLASSES AND OBJECTS

INTRODUCTION

C++ incorporates another user defined data type known as class. There is very little syntactical difference between structures and classes in C++. **Since class is a specially introduced data type in C++, most of the C++ programmers tend to use the structures for holding only data, and classes to hold both data and functions.**

Note: The only difference between a structure and a class in C++ is that, By default the members of a class are *private*, while, by default the members of a structure are *public*.

SPECIFYING A CLASS

A class is a way to bind the data and its associated functions together. It allows the data (and functions) to be hidden, if necessary, from external use. When defining a class, we are creating a new *abstract data type* that can be treated like any other built-in data type.

Generally, a class specification has two parts:

1. Class declaration
2. Class function definitions

The class declaration describes the type and scope of its members. The class function definitions describe how the class functions are implemented.

The general form of a class declaration is:

```
class class_name
{
    private:
        variable declarations;
        function declarations;
    public:
        variable declarations;
        function declarations;
    protected:
        variable declarations;
        function declarations;
};
```

- The class declaration is similar to a struct declaration. The keyword class specifies, that what follows is an abstract data of type *class_name*.
- The body of a class is enclosed within braces and terminated by a semicolon. The class body contains the declaration of variables and functions.
- These functions and variables are collectively called *class members*.
- They are usually grouped under two sections, namely, *private* and *public* to denote which of the members are *private* and which of them are *public*.

- The keywords `private`, `public` and `protected` are known as visibility labels. Note that these keywords are followed by a colon.
- The class members that have been declared as `private` can be accessed only from within the class.
- On the other hand, `public` members can be accessed from outside the class also.
- The members which are declared using `protected` can be accessed only by the member functions, friends of the class and also by the member functions derived from this class.
- The data hiding (using `private` declaration) is the key feature of object-oriented programming.
- The use of the keyword `private` is optional. **By default, the members of a class are private.**
- The variables declared inside the class are known as *data members* and the functions are known as *member functions*.
- Only the member functions can have access to the `private`, `protected` data members and `private` and `protected` functions.
- However, the `public` members (both functions and data) can be accessed from outside the class. This is illustrated in Fig. 2.1.
- The binding of data and functions together into a single class-type variable is referred to as **encapsulation**.

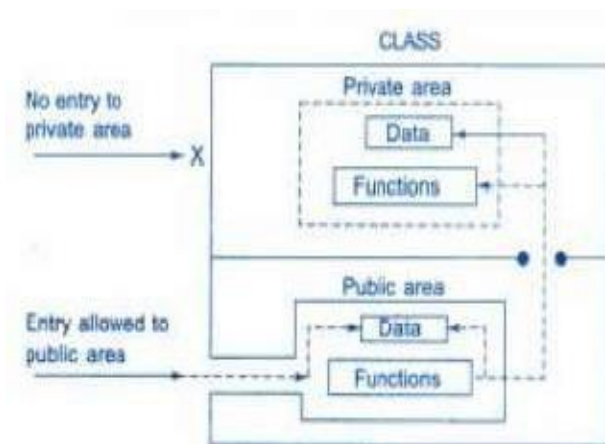


Figure 2.1: Data hiding in classes

A Simple Class Example

A typical class declaration would look like:

Example 1:

class item

```
{  
    int number;           // variables declaration  
  
    float cost;           // private by default
```

```
public:
void getdata(int a, float b);           // functions declaration
void putdata(void);                     // using prototype
};                                     // ends with semicolon
```

- We usually give a class some meaningful name, such as *item*. This name now becomes a new type identifier that can be used to declare *instances* of that class type.
- The class *item* contains two data members and two function members.
- The data members are private by default while both the functions are public by declaration.
- The function *getdata ()* can be used to assign values to the member variables *number* and *cost*, and *putdata()* for displaying their values.
- These functions provide the only access to the data members from outside the class. This means that the data cannot be accessed by any function that is not a member of the class *item*.
- Note that the functions are declared, not defined.
- Actual function definitions will appear later in the program. The data members are usually declared as private and the member functions as public.

Example 2: Let us declare a class for representation of bank

```
account.class account
{
    private: //implicit by
        default int accno;

        char name[20];
        char
        acctype[4]; int
        bal_amt;
    public: //member functions

        void get_data();
        void
        displaydata();
};
```

Example 3: A class to create student name, rollno, sex,

```
age.class student
{
    int rollno;
    char
    name[20];
    char gender;
    int age;
    public:
        void get_data();
```



```
void  
display_data();  
};
```

Access specifiers

An object is defined to be an instance of a class. Every data member of a class is specified by three levels of access protection for hiding data and function members internal to the class. The access specifiers define the scope of data. The access specifiers are indicated using the following keywords.

Private:

private access means a member data can only be accessed by the member function. Members declared under private are accessible only within the class. If no access specifier is mentioned, then by default, members are private.

```
Private:  
    int x;  
    float  
    y;
```

public:

public access means that members can be accessed by any function outside the class also.

```
class box  
{  
    int length;int height;  
    public : int width;  
    void set_height ( int i );void get_height();  
};
```

The keyword public identifies both class data and functions that constitute the public interface for the class. In the above program data member width is a public variable. set_height() and get_height() are member functions. The private variable height can be accessed only through the member functions of the class.

Protected:

The members which are declared using protected can be accessed only by the member functions, friends of the class and also by the member functions derived from this class. The members cannot be accessed from outside. The protected access specifier is therefore similar to private specifier.

Creating Objects

- Remember that the declaration of item as shown above does not define any objects of item but only specifies *what* they will contain.
- Once a class has been declared, we can create variables of that type by using the classname (like any other built-in type variable).

For example,

```
item x;           // memory for x is created
```

creates a variable x of type **item**. In C++, the class variables are known as *objects*. Therefore, x is called an object of type item. We may also declare more than one object

in one statement. Example:

```
item x, y, z;
```

The declaration of an object is similar to that of a variable of any basic type. The necessary memory space is allocated to an object at this stage. Note that class specification, like a structure provides *only a template* and does not create any memory space for the objects.

Objects can also be created when a class is defined by placing their names immediately after the closing brace, as we do in the case of structures.

That is to say, the definition

```
class item
{

    .....
    .....

    .....
}x,y, z;
```

would create the objects **x, y and z** of type **item**.

Accessing Class Members

As pointed out earlier, the private data of a class can be accessed only through the member functions of that class. The main() cannot contain statements that access **number and cost** directly. The following is the format for calling a member function:

```
object-name. function-name (actual1-arguments);
```

For example, the function call statement

```
x.getdata(100,75.5);
```

is valid and assigns the value 100 to **number** and 75.5 to **cost** of the object x by implementing the getdata() function. The assignments occur in the actual function.

Similarly, the statement

```
x.putdata() ;
```

would display the values of data members. Remember, a member function can be invoked only by using an object (of the same class). The statement like

```
getdata(100,75.5);
```

has no meaning. Similarly, the statement

```
x.number = 100;
```

is also illegal. Although x is an object of the type item to which number belongs, the number (declared private) can be accessed only through a member function and not by the object directly. It may be recalled that objects communicate by sending and receiving messages. This is achieved through the member functions. For example,

```
x.putdata();
```

sends a message to the object x requesting it to display its contents.

A variable declared as public can be accessed by the objects directly.

Example:

```
class xyz
{

    int
    x.; int
```

```

        y;
    public:
        int z;
};

.....
.....
.....
xyz p;
p.x = 0;                // error, x is private

p.z = 10;                //OK, z is public

.....
.....

```

DEFINING MEMBER FUNCTIONS

Member functions can be defined in two places:

- **Outside the class definition.**
- **Inside the class definition.**

It is obvious that, irrespective of the place of definition, the function should perform the same task. Therefore, the code for the function body would be identical in both the cases. However, there is a subtle difference in the way the function header is defined.

Outside the Class Definition

Member functions that are declared inside a class have to be defined separately outside the class. Their definitions are very much like the normal functions. They should have a function header and a function body. Since C++ does not support the old version of function definition, the *ANSI prototype* form must be used for defining the function header.

An important difference between a member function and a normal function is that a member function incorporates a membership 'identity label' in the header. This label tells the compiler which class the function belongs to. The general form of a member function definition is:

```

return-type class-name :: function-name (argument declaration)
{

```

Function body

```

}

```

The membership label `class-name::` tells the compiler that the function *function-name* belongs to the class *class-name*. That is, the scope of the function is restricted to the *class-name* specified in the header line. **The symbol `::` is called the *scope resolution operator*.**

For instance, consider the member functions `getdata()` and `putdata()` as discussed above. They may be coded as follows:

```

void item :: getdata( int a, float b)
{

    number =
    a; cost = b;

```

```

    }

    void item::putdata(void)
    {

        cout << "Number : " << number
        << "\n" ; cout << "cost :" << cost <<
        "\n";

    }

```

Since these functions do not return any value, their return type is void. Function arguments are declared using the ANSI prototype.

The member functions have some special characteristics that are often used in the program development. These characteristics are:

- Several different classes can use the same function name. The 'membership label' will resolve their scope.
- Member functions can access the private data of the class. A non-member function cannot do so. (However, an exception to this rule is a *friend* function discussed later.)
- A member function can call another member function directly, without using the dot operator.

Inside the Class Definition

Another method of defining a member function is to replace the function declaration by the actual function definition inside the class. For example, we could define the item class as follows:

```

class item
{

    int number;
    float cost;
    public:
        void getdata(int a, float b); //declaration of member
        function void putdata(void) // definition inside the
        class

        {

            cout << number << "\n":
            cout << cost << "\n";

        }

};

```

When a function is defined inside a class, it is treated as an in line function. Therefore all the restrictions and limitations that apply to an inline function are also applicable here. Normally, only small functions are defined inside the class definition.

Example Program with class

```
#include
<iostream.h>    class
item
{
    int    number;
    float cost;
public: void getdata(int a, float b);
    void putdata(void)
    {
        cout<<"Number:"<<number<<"\n";
        cout<<"Cost:"<<cost<<"\n";
    }
};

void item :: getdata(int a, float b)
{
    number =
    a;cost = b;
}
int main()
{
    item x;
    cout<<"\n        object
x"<<"\n"; x.getdata(100,
299.5); x.putdata();
    item y;
    cout<<"\n        object
y"<<"\n"; y.getdata(200,
175.50); y.putdata();
    return 0;
}
```

ARRAYS WITHIN A CLASS

Arrays can be declared as the members of a class. The arrays can be declared as private, public or protected members of the class.

Similar to other data members of a class, the memory space for an array is allocated when an object of the class is declared. In addition, different objects of the class have their own copy of the array. Note that the elements of the array occupy contiguous memory locations along with other data members of the object.

To understand the concept of arrays as members of class, consider the below example.

Example: A program to demonstrate the concept of arrays as class members.

```
#include<iostream.h>
const int size=5;
class student
{
int rollno;
int marks[size]; public: void getdata();
void tot_marks();
};
void student :: getdata ()
{
cout<<"\nEnter roll no: ";cin>>roll_no;
for(int i=0; i<size; i++)
{
cout<<"Enter marks in subject"<<(i+1)<<": ";cin>>marks[i] ;
}
}
void student :: tot_marks() //calculating total marks
{
int total=0;
for(int i=0; i<size; i++)total+= marks[i];
cout<<"\n\nTotal marks "<<total;
}
int main()
{
student stu;
stu.getdata() ;
stu.tot_marks() ;
return 0;
}
```

The output of the program is,

Enter roll no: 101

Enter marks in subject 1: 67

Enter marks in subject 2 : 54

Enter marks in subject 3 : 68

Enter marks in subject 4 : 72

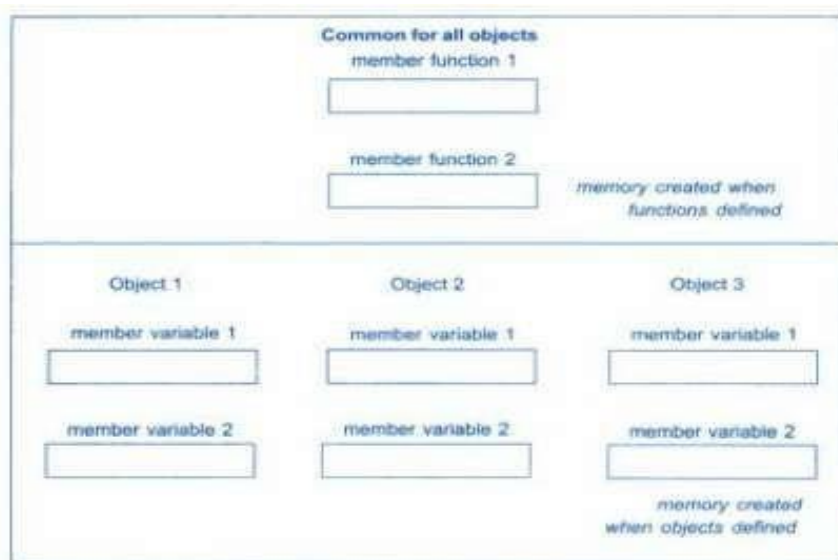
Enter marks in subject 5 : 82

Total marks = 343

In this example, an array marks is declared as a private member of the class student for storing a student's marks in five subjects. The member function tot_marks () calculates the total marks of all the subjects and displays the value.

MEMORY ALLOCATION FOR OBJECTS

The memory space for objects is allocated when they are declared and not when the class is specified. Actually, the member functions are created and placed in the memory space only once when they are defined as a part of a class specification. Since all the objects belonging to that class use the same member functions, no separate space is allocated for member functions when the objects are created. Only space for member variables is allocated separately for each object. Separate memory locations for the objects are essential, because



the member variables will hold different data values for different objects.

STATIC DATA MEMBERS

In C++ memory is allocated separately for each data member for different objects. So if you change the value of data member of class using one object will not affect the value of the same data member for other object of the same class. However sometimes it is required that all the objects of the same class share some common variables. This can be accomplished using the concept of static data member. **We can declare static data members using static keyword.**

Static data member having several characteristics which are given below:

- Hence all the object of the same class share static data member memory is allocated only once to the static data member. It remains common for all the objects of the same class.
- Static data member is initialized to 0 when first object of the class is created.
- Static data member is declared in the class but it must be defined outside the class using class name and scope resolution operator (::) because memory allocation for static data member of the class is performed different then normal data member of the class and it is not the part of class object.

Example:

```
#include<iostream.h>
#include<conio.h>
class item
{
static int count;int number;
public:
void getdata(int a)
{
number=a'count ++;
}
void DisplayCounter()
{
cout<<"count:"<<count<<endl;
}
};
int item::count;
int main()
{
item a,b,c;
a.DisplayCounter();
b.DisplayCounter();
c.DisplayCounter();
return 0;
}
```

Output:

```
Count: 1
Count: 2
Count: 3
```

Note:

Notice the following statement in the program:

int item :: count; // definition of static data member

Note that the type and scope of each static member variable must be defined outside the class definition. This is necessary because the static data members are stored separately rather than as a part of an object. Since they are associated with the class itself rather than with any class object, they are also known as class variables.

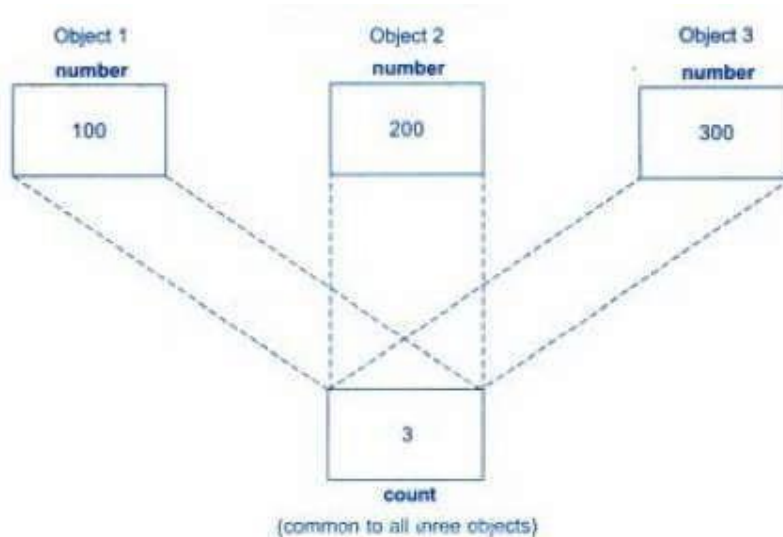
The static variable count is initialized to zero when the objects are created. The count is incremented whenever the data is read into an object. Since the data is read into objects three times, the variable count is incremented three times. Because there is only one copy of count shared by all the three objects, all the three output statements cause the value 3 to be displayed.

While defining a static variable, some initial value can also be assigned to the variable. For

instance, the following definition gives count the initial value 10.

```
int item :: count= 10;
```

Below figure shows how a static variable is used by the objects.



STATIC MEMBER FUNCTIONS

Like static member variable, we can also have static member functions. A member function that is declared **static** has the following properties:

- A static function can have access to only other static members (functions or variables) declared in the same class.
- A static member function can be called using the class name (instead of its objects) as follows:

```
class-name:: function -name;
```

Below program illustrates the implementation of these characteristics. The static function `showcount()` displays the number of objects created till that moment. A count of number of objects created is maintained by the static variable `count`.

The function `showcode()` displays the code number of each object.

```
#include <iostream.h>#include<conio.h> class test
{
int code;
static int count;    // static member variable

public:
void setcode(void)
{
```

```

code = ++count;
}
void showcode(void)
{
cout << "object number: " << code << "\n";
}
static void showcount(void) // static member function
{
cout << "Count: " << count << "\n";
}
};
int test :: count;int main()
{
clrscr(); test t1, t2;
t1.setcode();
t2. setcode();
test :: showcount(); //accessing static functiontest t3:
t3 .setcode();
test : : snowcount ();t1. showcode():
t2 .showcode():t3.showcode(): return 0; getch();
}

```

Output of Program:

```

count: 2
count: 3
object number: 1
object number: 2
object number: 3

```

Note that the statement

code = ++count;

is executed whenever setcode() function is invoked and the current value of count is assigned to code. Since each object has its own copy of code, the value contained in code represents a unique number of its object.

Remember, the following function definition will not

```

work: static void showcount()
{
    cout << code; //code is not static
}

```

ARRAYS OF OBJECTS

We know that an array can be of any data type including struct. Similarly, we can also have arrays of variables that are of the type class. Such variables are called **arrays of objects**. Consider the following class definition:

```

class employee

```

```

{
    char
    name[10];
    float    age;
    public:

        void
        getdata(void); void
        putdata(void);
};

```

The identifier employee is a user defined data type and can be used create objects that relate to different categories of the employees. Example:

```

employee manager[3];    //array    of
manageremployee foreman[15];    //array of
foremanemployee worker[75] ;    //array of
worker

```

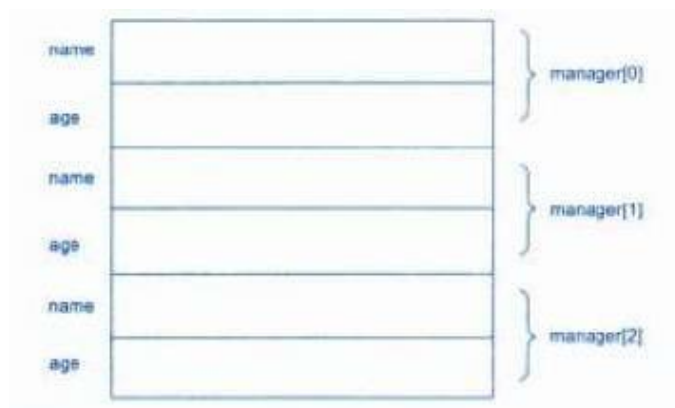
The array manager contains three objects(managers), namely, manager[0], manager[1], and manager [2], of type employee class. Similarly, the foreman array contains 15 objects (foremen) and the worker array contains 75 objects (workers).

Since an array of objects behaves like any other array, we can use the usual array-accessing methods to access individual elements, and then the dot member operator to access the member functions. For example, the statement

```
manager[i] .putdata( );
```

will display the data of the ith element of the array manager. That is, this statement requests the objectmanager[i] to invoke the member function putdata().

An array of objects is stored inside the memory in the same way as a multi-dimensional array. The array manager is represented in below figure. Note that only the space for data items of the objects is created. Member functions are stored separately and will be used by all the objects.



Example program for arrays of objects

```
#include <iostream.h>
class employee
{
char name[30]; // string as class member
float age;
public:
void getdata(void);
void putdata(void);
};
void employee::getdata(void)
{
cout << "Enter name:"; cin >> name;
cout << "Enter age:";
cin >> age;
}
void employee::putdata(void)
{
cout << "Name: "<< name<< "\n"; cout << "Age: "<< age<< "\n";
}
const int size=3; int main()
{
employee manager[size]; for(int i=0; i<size; i++)
{
cout << "\nDetails of manager" << i+1 << "\n"; manager[i].getdata();
}
cout << "\n"; for(i=0; i<size; i++)
{
cout << "\nManager" << i+1 << "\n"; manager[i].putdata();
}
return 0;
}
```

This being an interactive program, the input data and the program output are shown below:

Interactive input

Details of manager1

Enter name: xxx

Enter age: 45

Details of manager2

Enter name: yyy

Enter age: 37

Details of manager3

Enter name: zzz

Enter age: 50

Program output

Manager1

Name: xxx
Age: 45
Manager2
Name: yyy
Age: 37
Manager3
Name: zzz
Age: 50

OBJECTS AS FUNCTION ARGUMENTS

Like any other data type an object may be used as a function argument. This can be done in two ways:

- A copy of the entire object is passed to the function.
- Only the address of the object is transferred to the function.

The first method is called ***pass-by-value***. Since a copy of the object is passed to the function, any changes made to the object inside the function do not affect the object used to call the function. The second method is called ***pass by reference***. When an address of the object is passed, the called function works directly on the actual object used in the call. This means that any changes made to the object inside the function will reflect in the actual object. The pass-by reference method is more efficient since it requires to pass only the address of the object and not the entire object..

Program illustrates the use of objects as function arguments. It performs the addition of time in the hour and minutes format.

```
#include<iostream.h>class time
{
int hours;
int minutes;
public:
void gettime(int h, int m)
{
hours =h; minutes= m;
}
void puttime(void)
{
cout << hours <<" hours and ";
cout << minutes << " minutes" << "\n";
}
void sum(time,t1); // declaration with objects as arguments
};
void time :: sum(time t1 , time t2) // t1 , t2 are Objects
{
minutes = t1.minutes + t2.minutes;hours = minutes/60; minutes=minutes%60 ;
hours = hours + t1.hours + t2.hours;
}

int main()
{
```

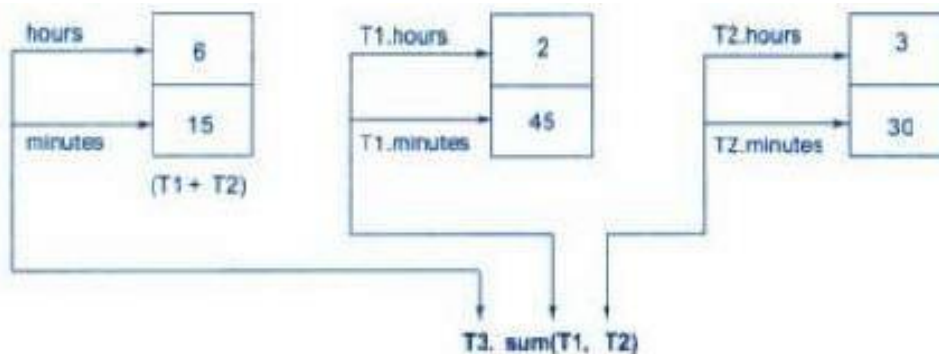
```

time T1, T2, T3;
T1.gettime(2,45;
T2.gettime(3,30;
T3.sum(T1,T2);
cout << "T1=";
T1.puttime();
cout << "T2=";
T2.puttime();
cout << "T3=";
T3.puttime(); return 0;
}

```

The output of Program would be:

T1 = 2 hours and 45
minutes T2 = 3 hours and
30 minutes T3 = 6 hours
and 15 minutes



FRIEND FUNCTIONS

We have seen that private and protected members of a class cannot be accessed from outside the class in which they are declared. In other words non-member function does not have access to the private data members of a class.

But there could be a situation where two classes must share a common function. C++ allows the common function to be shared between the two classes by making the common function as a friend to both the classes, thereby allowing the function to have access to the private data of both of these classes.

A friend function is a non-member function that is a friend of a class. The friend function is declared within a class with the prefix **friend**. But it should be defined outside the class like a normal function without the prefix friend. It can access public data members like non-member functions.

A friend function possesses certain special characteristics:

- A friend function although not a member function, has full access right to the private and protected members of the class.
- A friend function cannot be called using the object of that class. It can be invoked like any normal function.

- They are normal external functions that are given special access privileges.
- It cannot access the member variables directly and has to use an object name.membername. It can be invoked like a normal function without the help of any object.
- The function is declared with keyword friend. But while defining friend function it does not use either keyword friend or :: operator.
- Unlike member functions, it cannot access the member names directly and has to use an objectname and dot membership operator with each member name.(e.g.A.x).
- It can be declared either in the public or the private part of a class without affecting its meaning.
- Usually, it has the objects as arguments.

Syntax:

```
class class_name
{
public:
    friend void function1(void);
    friend returntype_specifier function_name(arguments);
};
```

Example :

```
class base
{
    int val1, val2;
public:
    void getdata()
    {
        cout<<"Enter two values:";cin>>val1>>val2;
    }
    friend float mean(base ob);
};

float mean(base ob)
{
    return float(ob.val1+ob.val2)/2;
}
```

In the above example mean() is declared as friend function that computes mean value of two numbers that are input using getdata() function.

Program : To indicate the use of friend

function. #include <iostream.h>

class myclass

{

private:

int a,b;

public:

void set_val(int i, int j);

friend int add(myclass

obj);

void myclass::set_val(int i,int j)

{

a = i;b = j;

}

int add(myclass obj)

{

return (obj.a+obj.b);

}

int main()

{

myclass object;

object.set_val(34, 56);

cout << "Sum of 34 and 56 is "<<add(object)<<endl;

return 0;

}

Output

Sum of 34 and 56 is 90

In the above program segment the function add() is a friend of class myclass. Since it is not a member of any class, it cannot be called like an object. Since it is a non-member function, add() should be declared inside a class under public or private or protected access specifier.

RETURNING OBJECTS

A function cannot only receive objects as arguments but also can return them. The example Program illustrates how an object can be created (within a function) and returned to anotherfunction.

```
#include <iostream.h>
```

```
class matrix
```

```
{
```

```
    int    m[3][3];
```

```
    public:
```

```
    void read()
```

```
    {
```

```
        cout<<"Enter the elements of the 3x3
```

```
matrix:\n";int i,j;
```

```
for(i=0;i<3;i++)
```

```
    for(j=0;j<3;j++)
```

```
    {
```

```
        cout<< "m[<<i<<"] [<<j<<"]=";
```

```
        cin>>m[i][j];
```

```
    }
```

```
}
```

```
    void display()
```

```
    {
```

```
        int            i,j;
```

```
        for(i=0;i<3;i++)
```

```
        for(j=0;j<3;j++)
```

```
        {
```

```
            cout<<m[i][j];
```

```
        }
```

```
}
```

```
    friend matrix transpose(matrix);
```

```
};
```

```
matrix transpose(matrix m1)
```

```
{
```

```
    matrix m2;
```

```
    int i,j;
```

```
    for(i=0;i<3;i++)
```

```
        for(j=0;j<3;j++)
```

```
            m2.m[i][j]=m1.m[j][i];
```

```
    return (m2);
```

```
}
```

```
void main()
```

```
{
```

```
    matrix mat1,mat2; mat1.read(); mat1.display();
```

```

        mat2=transpo
        se(mat1);
        mat2.display(
        ); getch();
    }

```

Output

```

m[0][0]=1
m[0][1]=2
m[0][2]=3
m[1][0]=4
m[1][1]=5
m[1][2]=6
m[2][0]=7
m[2][1]=8
m[2][2]=9

```

```

1  2  3
4  5  6
7  8  9

```

```

1  4  7
2  5  8
3  6  9

```

INLINE FUNCTIONS

A function call generally involves the complex process of invoking a function, passing parameters to the function, allocating storage for local variables, thereby using extra time and memory space. It is possible to avoid these overheads of a function by using inline function. The inline function is a short function. Compiler replaces a function call with the body of the function.

The keyword inline is used to define inline functions. The inline function consists of function call along with function code and the process is known as expansion.

- Inline functions definition starts with keyword inline.
- The inline functions should be defined before all functions that call it.
- The compiler replaces the function call statement with the function code itself (expansion) and then compiles the entire code.
- They run little faster than normal functions as function calling overheads are saved.

Advantages of inline functions

- The inline member functions are compact function calls.
- Therefore the size of the object code is considerably reduced.
- The speed of execution of a program increases.
- Very efficient code can be generated.
- The readability of the program increases.

Program: Finding the square of a number using inline functions. #include <iostream.h>

```

class sample
{
    public: inline int square (int a)
        {
            return(a*a);
        }
}

```

```
int main( )
{
    int x,
    y;
    x=square(5);
    ;
    cout<<"Square of 5 =
    "<<x<<endl;    y=square(10);
    cout<<"Square of 10 =
    "<<y<<endl; return 0;
}
```

Output

Square of 5 =
25

Square of 10 = 100

In the above example square() is an inline function that finds the square of a number.

Multiple Choice Questions

REMEMBERING (LEVEL 1)

1) What is a collection of set of data and functions?

A. Object

B. B. Encapsulation

C. C. Dynamic Binding

D. D. Polymorphism

2) What is the wrapping up of data and functions into a single unit is known as?

A. Dynamic Binding

B. Polymorphism

C. Inheritance

D. Encapsulation

3) What is a template representing a group of objects that share common properties and relationships called?

A. Object

B. Class

C. Inheritance

D. Encapsulation

4) Which one of the below mentioned facilitate objects of one class to acquire the properties of objects of another class.

A. Polymorphism

B. Inheritance

C. Message Passing

D. Encapsulation

5) The code associated with a given procedure call is not known until the time of call at run-time, this is due to?

A. Inheritance

B. Dynamic Binding

C.

Polymorphism

D. Data Abstraction

6) Which operator is used to define the member functions outside the class?

A. \$

B. ::

C. &

D. *

7) Which of the following term is used for a function defined inside a class?

A. Member Variable

B. Member Function

C. Class Function

D. Static Function

8) Which function can be invoked like a normal function without the help of any object?

A) constant member function

B) private member function

C) static member function

D) friend function

9) Although not a member function, which of the below mentioned function has full access rights to the private members of the class.

A) constant member function

B) private member function

C) protected member function

D) friend function

10) In which method of passing arguments using an object as a function argument, a copy of the entire object is passed to the function

A) pass-by-value

B) pass-by-reference

C) pass-by-variable

D) pass-by-function

Skill (Level 4)

11) Can you identify an object-based programming language among these?

B. C++

B. Java

C. ADA

D. Smalltalk

12) Judge which of the below statements are True?

i. OOP software is not having set standards.

ii. To convert a real-world problem into an object oriented model is easy.

iii. The classes are overly generalized.

- A. i and ii B.
All of the above
C. i and iii D. i only

13) Identify which is the access specifier for the members of a class when not specifically mentioned or specified

- A. Public **B. Private** C. Protected D. Static

14) Can you judge which kind of access means that members can be accessed by any function outside the class also.

- A. **Public** B. Private C. Protected D. Static

15) Predict the correct format for calling a member function:

- A. Object-name. function-name (actual-arguments);**
B. Object-name: function-name (actual-arguments);
C. Object-name= function-name (actual-arguments);
D. Object-name& function-name (actual-arguments);

16) Select the valid statement to access, If getdata(int a, float b) is a member function of the class,

- A. getdata (100, 25.5) B. **x.getdata(100, 25.5)** C. x =getdata(100,25.5) D.getdata()

17) Can you suggest, how should the data member be declared in a class so as to maintain values common to the entire class

- A. Private B. public **C. static** D. protected

18) Which is most appropriate comment on following class definition? P

```
class Student
{
    int a;
    public: float a;
};
```

- a. Error: Same variable can't be used twice.**
b. Error: Public must be declared first
c. Error: Data types are different for same variable.
d. It is correct

19) Identify which of the below mentioned syntax for class definition is wrong

- A) class student{ };
B) student class{ };
C) class student {public: student(int a){ }
};
D) class student{ student(int a){ }
};

20) Identify which feature of OOP is used in below scenario- "If a function can perform more than 1 type of tasks, where the function name remains same"

- A) Encapsulation
B) Inheritance
C) Polymorphism
D) Abstraction

21) Can you Judge which of the following statements regarding inline functions is incorrect?

- A. It speeds up execution B. It slows down execution
C. It increases the code size **D. Both A and C**

22) Can you predict, what is correct about the static data member of a class?

- A. A static member function can access all the data members of a class.
B. Static member remains common for all the objects of the same class.
C. Static data member is not initialized to 0 when first object of the class is created.

D. Static data member is can't be declared inside the class.

23) Identify which of the below can be used to call a static member function instead of objects.

A) variable name

B) function name

C) Class name

D) object name

24) Identify which of the following is the only technical difference between Structures and classes in C++?

A. Members are by default protected in structures but private in classes.

A. Members are by default private in structures but public in classes.

B. Members are by default public in structures but private in classes.

C. Members are by default public in structures but protected in classes.

25) Select an option which is true about static Member function,

A. It can access any other member function and member variable.

B. It can access only static member variables and member functions

C. It can be only called through the object of the class

D. It returns only static data

II Answer The Following Questions

1. What is meant by object-oriented programming? How can you distinguish Object oriented programming from the procedure-oriented programming ?

2. What is meant by a class? Can you explain class definition with an example.

3. Outline the basic concepts of oops.

4. What can you say about different Access specifiers used in a class.

5. What is meant by friend function? List out the characteristics of friend functions?

6. Explain in brief about inline function with example

1. Can you identify the advantages OOPs?

2. Identify some of the limitations of object oriented programming.

3. Can you identify some areas where object oriented programming is needed in real life applications.

4. How is a member function of a class defined? Explain.

5. Illustrate with an example how an array of objects can be defined?

6. Illustrate with an example how can objects be passed as function arguments

UNIT-II
CONSTRUCTORS, DESTRUCTORS AND OPERATOR OVERLOADING
CHAPTER 3: CONSTRUCTORS AND DESTRUCTORS

CONSTRUCTORS

A constructor is a special member function whose task is to initialize the objects of its class. It is special because its name is the same as the class name. The constructor is invoked whenever an object of its associated class is created. It is called constructor because it constructs the values of data members of the class.

A constructor is declared and defined as follows;

// class with a constructor

```
class integer
{
int    m,    n;
public:
integer(void) ; //constructor declared
-----
};
integer :: integer(void) // constructor defined
{
m=0; n=0;
}
```

When a class contains a constructor like the one defined above, it is guaranteed that an object created by the class will be initialized automatically. For example, the declaration

```
integer int1;           //object int1 created
```

not only creates the object int1 of type integer but also initializes its data members m and n to zero. There is no need to write any statement to invoke the constructor function (as we do with the normal member functions). If a 'normal' member function is defined for zero initialization, we would need to invoke this function for each of the objects separately.

Special Characteristics:

The constructor functions have some special characteristics. These are:

- They should be declared in the public section.
- They are invoked automatically when the objects are created.
- They do not have return types, not even void and therefore, and they cannot return values.
- They cannot be inherited, though a derived class can call the base class constructor.
- Like other C++ functions, they can have default arguments.
- Constructors cannot be virtual.
- We cannot refer to their addresses.
- An object with a constructor (or destructor) cannot be used as a member of a union.
- They make 'implicit calls' to the operators new and delete when memory allocation is required.

Remember, when a constructor is declared for a class, initialization of the class objects becomes mandatory.

Example program:

```
#include <iostream.h>
#include <iomanip.h>
class counter
{
private:
int a;
public:
        counter()
        {
a = 0;
}
void inc_count()
{
a++;
}
int get_count()
{
return a;
}
};
void main()
{
counter c1, c2;
cout<<"c1    =    "<<c1.get_count();
cout<<"c2    =    "<<c2.get_count();
c1.inc_count();
c2.inc_count();
cout<<"c1 = "<<c1.get_count()<<endl; cout<<"c2 = 
"<<c2.get_count()<<endl;
}
```

The above code segment contains a constructor with the name counter(), which is same as the class name. This constructor initializes the class variable a to 0. When, in main(), an object for the counter class is defined will invoke or call the constructor counter() and initialize the variable a to

0. i.e., in main() we are defining two instances for the class counter(), c1 and c2. Now, the constructor counter will be called twice automatically, once for object c1 and once for object c2.

TYPES OF CONSTRUCTORS

Constructors can be divided into 3 types,

- Default Constructor.
- Parameterized Constructor.
- Copy Constructor.

Default Constructor

constructor that accepts no parameters is called the *default constructor*. The default constructor for class A is A::A(). If no such constructor is defined, then the compiler supplies a default constructor. Therefore a statement such as

A a;

invokes the default constructor of the compiler to create the object a. #include<iostream.h>

#include<iomanip.h>

```
class
x
{
    private:
    int a, b;
public:
    x( )
    {
        a=10, b=20;
    }
void display()
{
    cout<<a<<setw(5)<<b<<endl;
}
};
void main()
{
    x obj1,obj2; obj1.display();
obj2.display();
}
```

Output:

```
10 20
10 20
```

Parameterized Constructors

A constructor with one or more parameters is called a parameterized constructor. The advantage of a parameterized constructor is that you can initialize each instance of the class to different values.

The constructor integer(), defined above, initializes the data members of all the objects to zero. However, in practice it may be necessary to initialize the various data elements of different objects with different values when they are created. C++ permits us to achieve this objective by passing arguments to the constructor function when the objects are created.

The constructor integer() may be modified to take arguments as shown below: class integer

```
{
    int m, n;
public:
    integer (int x, int y); //parameterized constructor
    .....
    .....
};
integer :: integer(int x, int y)
{
    m=x; n=y;
}
```

When a constructor has been parameterized, the object declaration statement such as `integer int1;`

may not work. We must pass the initial values as arguments to the constructor function when an object is declared. This can be done in two ways:

- By calling the constructor explicitly.
- By calling the constructor implicitly.

The following declaration illustrates the first method:

```
integer int1 = integer(0,100); // explicit call
```

This statement creates an integer object `int1` and passes the values 0 and 100 to it. The second is implemented as follows:

```
integer int1(0,100); // implicit call
```

This method, sometimes called the shorthand method, is used very often as it is shorter, looks better and is easy to implement.

Remember, when the constructor is parameterized, we must provide appropriate arguments for the constructor.

Below program demonstrates the passing of arguments to the constructor functions.

```
#include<iostream.h>
```

```
class integer
```

```
{
```

```
int      m,n;
```

```
public:
```

```
integer(int, int);           //constructor
```

```
declared void display( )
```

```
{
```

```
cout << " m =" << m << "\n": cout << " n =" <<
```

```
n << "\n";
```

```
}
```

```
};
```

```
integer :: integer(int x, int y)
```

```
{
```

```
m=x; n=y;
```

```
}
```

```
void main( )
```

```
{
```

```
integer int1(0,100);        //constructor called implicitly
```

```
integer int2=integer(25,75); //constructor called explicitly
```

```
cout<<"Object1"<<"\n";
```

```
int1.display( );
```

```
cout<<"Object2"<<"\n";
```

```
int2.display( );
```

```
}
```

Output:

```
Object1
```

```
m=0
```

```
n=100
```

```
Object2
```

```
m=25
```

```
n=75
```

The constructor functions can also be defined as inline functions.

Example:

```
class integer
```

```
{
```

```
int m, n;
```

```
public:
```

```
integer(int x, int y) // Inline constructor
```

```
{
```

```
    m = x;
```

```
    n=y;
```

```
}
```

```
.....
```

```
.....
```

```
}
```

```
;
```

The parameters of a constructor can be of any type except that of the class to which it belongs. For example,

```
class A
```

```
{
```

```
public:
```

```
    A(A);
```

```
}
```

```
;
```

is illegal. However, a constructor can accept a *reference* to its own class as a parameter.

Thus, the

statement class A

```
{
```

```
public:
```

```
    A(A&);
```

```
}
```

```
;
```

is valid .

In such cases, the constructor is called the *copy constructor*.

Copy Constructor

The constructor which creates an object by copying variables from another object is called a copy constructor. The purpose of a copy constructor is to initialize a new instance to the values of an existing instance.

For example, the statement **integer I1;**

integer I2(I1);

would define the object I2 and at the same time initialize it to the values of I1. Another form of this statement is

integer I2 =I1;

The process of initializing through a copy constructor is known as *copy initialization*.

Remember, the statement

I2=I1;

will not invoke the copy constructor. However, if I1 and I2 are objects, this statement is legal and simply assigns the values of I1 to I2, member-by-member. This is the task of the overloaded assignment operator(=).

A copy constructor takes a reference to an object of the same class as itself as an argument. Let us consider a simple example of constructing and using a copy constructor as shown in below program.

```
#include
<iostream.h>   class
code
{
int      id;
public:
code() { }           //default      constructor
code(int a)         //parameterized
constructor
{
id = a ;
}
code(code & x)       //copy constructor
{
id=x. id;
}
void display(void)
{
cout << id;
}
};
void main( )
{
code A (I00); // object A is created and initialized
code B(A);    // copy constructor called
code C = A;   //copy constructor called
again code D; // D is created, not
initialized D= A; // copy constructor not
called cout << "\n id of A: ";
A.display( );
cout << "\n id of B:" ;
```

```
B.display();  
cout << "\n id of C:";  
C.display() ;  
cout << "\n id or D:";  
D.display();  
}
```

The output of Program is shown below:

```
id of A:  
100 id of  
B: 100 id  
of C: 100  
id of D:  
100
```

Note: **A reference variable has been used as an argument to the copy constructor.** We cannot pass the argument by value to a copy constructor.

When no copy constructor is defined, the compiler supplies its own copy constructor.

MULTIPLE CONSTRUCTORS IN A CLASS

Here, we have used two kinds of constructors. They are:

```
integer( ) ;           //No arguments  
integer(int, int ) ; // Two arguments
```

In the first case, the constructor itself supplies the data values and no values are passed by the calling program, In the second case, the function call passes the appropriate values from main(). C++ permits us to use both these constructors in the same class. For example, we could define a class as follows: class integer

```
{  
int m, n;  
public:  
  
integer( )           //constructor 1  
{  
m=0; n=0;  
}  
integer(int a, int b) //constructor 2  
{  
m= a; n= b;  
}  
integer(integer & i ) //constructor 3  
{  
m= i.m;  
n = i.n ;  
}
```

This declares three constructors for an integer object.. The first constructor receives no arguments, the second receives two integer arguments and the third receives one integer object as an argument. For example, the declaration

integer I1;

would automatically invoke the first constructor and set both m and n of I1 to zero. The statement integer I2(20,40) ;

would call the second constructor which will initialize the data members m and n of I2 to 20 and 40 respectively. Finally, the statement

integer I3(I2) ;

would invoke the third constructor which copies the values of I2 into I3. In other words, it sets the value of every data element of I3 to the value of the corresponding data element of I2. As mentioned earlier, such a constructor is called the *copy constructor*. When more than one constructor function is defined in a class, we say that the **constructor is overloaded**.

Program below shows the use of overloaded constructors.

```
#include
<iostream.h>   class
complex
{
float   x.   y;
public:
complex() { } //constructor no arg
complex (float a) // constructor-one arg
{
x =y = a;
}
complex( float real, float imag) // constructor-two args
{
x =real ; y =imag;
}
friend complex sum(complex, complex); friend void
show(complex):
};
complex sum{complex c1, complex c2)
{
complex c3:
c3.x =c1.x + c2 .x; c3.y
=   c1.y   +   c2.y;
return(c3);
}

void show(complex c)
{
cout << c.x << “ + “ << c.y <<”\n”;
}
void main()
{
complex A(2.7, 3.5):
complex B(1.6):
complex C:
```

```

C=sum(A, B);
cout <<"A="; show(A);
cout << "B = " ;
show(B);
cout <<"C=";
show(C);
// Another way to give Initial values (second method)
complex P,Q, R;
P = complex(2.5,3.9);
Q = complex(1.6,2.5); R =
sum(P,Q);
cout <<"\n"; cout <<
"p = ";
show(P) ;
cout << "Q = ";
show(Q);
cout << "R = ";
show(R);
}

```

The output of Program would be:

```

A= 2.7+1.5i
B = 1.6 + 1.6i
C = 4. 3 +
5.1i p = 2.5 +
3.9i Q = 1. 6
+ 2. 5i R = 4.
1 + 6.4i

```

There are three constructors in the class complex. The first constructor, which takes no arguments, is used to create objects which are not initialized; the second, which takes one argument, is used to create objects and initialize them; and the third, which takes two arguments, is also used to create objects and initialize them to specific values.

CONSTRUCTORS WITH DEFAULT ARGUMENTS

It is possible to define constructors with default arguments. For example, the constructor `complex()` can be declared as follows:

```
complex(float real , float imag=0) ;
```

The default value of the argument `imag` is zero. Then, the

```
statement complex C(5.0) ;
```

assigns the value 5.0 to the real variable and 0.0 to `imag` (by default). However, the statement `complex C(2.0,3.0);`

assigns 2.0 to `real` and 3.0 to `imag`. The actual parameter, when specified, overrides the default value. As pointed out earlier, the missing arguments must be the trailing ones. It is important to distinguish between the default constructor `A::A()` and the default argument constructor `A::A(int`

`= 0)`. The default argument constructor can be called with either one argument or no arguments. When called with no arguments, it becomes a default constructor. When both these forms are used in a class, it causes ambiguity for a statement such as

A a;

The ambiguity is whether to 'call' A::A() or A::A(int = 0).

DESTRUCTORS

A destructor, as the name implies is used to destroy the objects that have been created by a constructor. Like a constructor, the destructor is a member function whose name is the same as the class name but is preceded by a **tilde(~)**. For example, the destructor for the class integer can be defined as shown below:

```
~integer() { }
```

- A destructor never takes any argument nor does it return any value.
- It will be invoked implicitly by the compiler upon exit from the program (or block or function as the case may be) to clean up storage that is no longer accessible.
- It is a good practice to declare destructors in a program since it releases memory space for future use.

Whenever new is used to allocate memory in the constructors, we should use delete to free that memory. For example, the destructor for the matrix class discussed above may be defined as follows:

```
matrix : : ~matrix()  
{  
for(int i=0; i<d1; i++) delete  
p[ i ] ;  
delete p;  
}
```

This is required because when the pointers to objects go out of scope, a destructor is not called implicitly.

Example program:

```
#include<iostream.h>  
#include<conio.  
h>  
#include<string.  
h> class book  
{  
int accno;  
char author_name[20]; char  
title[20];  
int year;  
char pub_name[20];  
float cost;  
public:  
book()  
{  
accno=101;  
strcpy(author_name,"balaguruswamy");  
strcpy(title,"oops");  
year=1990;  
strcpy(pub_name,"Mc Graw Hill");  
cost=400;
```



```
}

void display()
{
    cout<<"accession      number:"<<accno<<endl;
    cout<<"author_name    :"<<author_name<<endl;
    cout<<"title          :"<<title<<endl;
    cout<<"year            :"<<year<<endl;
    cout<<"pub_name        :"<<pub_name<<endl;
    cout<<"cost            :"<<cost<<endl;
}

~book()
{

    cout<<"object is destroyed";

}

};

void main()
{
    book      b;
    clrscr();
    b.display();
    getch();
}
```

CHAPTER 4: OPERATOR OVERLOADING AND TYPE CONVERSIONS

INTRODUCTION

Operator overloading is one of the main exciting features of C++ language. It is an important technique that has enhanced the power of extensibility of C++. We have stated more than once that C++ tries to make the user-defined data types behave in much the same way as the built-in types. For instance, C++ permits us to add two variables of user-defined types with the same syntax that is applied to the basic types. This means that C++ has the ability to provide the operators with a special meaning for a data type. **The process of making an operator to exhibit different behaviours in different instances is known as operator overloading.**

Operator overloading provides a flexible option for the creation of new definitions for most of the C++ operators. **We can overload (give additional meaning to) all the C++ operators except the following:**

- Class member access operators(. , .*).
- Scope resolution operator (::).
- Size operator (sizeof).
- Conditional operator (?:).

Although the *semantics* of an operator can be extended, we cannot change its *syntax*, the grammatical rules that govern its use such as the number of operands, precedence and associativity. For example, the multiplication operator will enjoy higher precedence than the addition operator. Remember, when an operator is overloaded, its original meaning is not lost. For instance, the operator +, which has been overloaded to add two vectors, can still be used to add two integers.

DEFINING OPERATOR OVERLOADING

To define an additional task to an operator, we must specify what it means in relation to the class to which the operator is applied. This is done with the help of a special function, called *operator function*, which describes the task. The general form of an operator function is:

```
return type classname :: operator op(arglist)
{
    function body // task defined
}
```

where *return type* is the type of value returned by the specified operation and *op* is the operator being overloaded. The *op* is preceded by the keyword *operator*. *operator op* is the function name.

Operator functions must be either member functions (non-static) or friend functions. A basic difference between them is that a friend function will have only one argument for unary operators and two for binary operators, while a member function has no arguments for unary operators and only one for binary operators. This is because the object used to invoke the member function is passed implicitly and therefore is available for the member function. This is not the case with friend functions. Arguments may be passed either by value or by reference. Operator functions are declared in the class using prototypes as follows:

```
void operator+();
void operator-( );
friend int operator+ (int, int);
```

```
friend void operator-
(void); int operator==( );
friend int operator= (int, int)
```

The process of overloading involves the following steps:

1. Create a class that defines the data type that is to be used in the overloading operation.
2. Declare the operator function operator *op()* in the public part of the class. It may be either a member function or a friend function.
3. Define the operator function to implement the required operations. Overloaded operator functions can be invoked by expressions such as

+ x or x + for unary operators and

x +y

for binary operators. + x (or x +) would be interpreted as operator + (x) for friend functions.

Similarly, the expression x+y would be interpreted as either x.operator + (y) in case of member functions, or operator + (x,y) in case of friend functions.

OVERLOADING UNARY OPERATORS

Let us consider the unary minus - operator. A minus operator when used as a unary, takes just one operand. We know that this operator changes the sign of an operand when applied to a basic data item. We will see here how to overload this operator so that it can be applied to an object in much the same way as is applied to an int or float variable. The unary minus when applied to an object should change the sign of each of its data items.

Below program shows how the unary minus operator is overloaded.

```
#include <iostream.h>
class space
{
int x; int
y; int z;
public:
void getdata(int a, int b, int
c); void display(void);
void operator-(); //overload unary minus
};
```

```

void space :: getdata (int a, int b, int c)
{
x = a; y = b;
z = c;
}

void space :: display(void)
{
cout <<" x ="<< x; cout
<<" y ="<<y; cout <<" z
="<< z;
}

void space :: operator-()
{
x = -x;
y = -y;
z = -z;
}

void main()
{
space S;
S.getdata(10,    -20,    30);
cout<<"S:";
S.display();
-S;                //activates    operator-()
function cout<<"-S:";
S.display();
}

```

The above program produces the following output:

```

S :  x=10  y=- 20  z=30
S:  x = - 10  y=20  z=-
30

```

Note:

The function operator -() takes no argument. It changes the sign of data members of the object S. Since this function is a member function of the same class, it can directly access the members of the object which activated it.

OVERLOADING BINARY OPERATORS USING MEMBER FUNCTION We have just seen how to overload an unary operator. The same mechanism can be used to overload a binary operator.

A statement like,

C = sum(A, B); // functional notation.

can be used. The functional notation can be replaced by a natural looking

expression C = A + B; // arithmetic notation

by overloading the + operator using an operator+() function. The Program below illustrates how this is accomplished.

```

#include <iostream.h>
    class complex
    {
float x; float
y; public:
        complex() { }
        complex(float real, float imag)
        {
x = real;
y=imag;
}
        complex      operator+(complex);      void
display(void);
};
        complex complex :: operator+(complex c)
        {
        complex temp; temp.x
        = x + c.x; temp.y = y
        + c.y; return(temp);
        }
        void complex :: display(void)
        {
        cout << x << " + " << y << "i\n";
        }
        void main()
        {
        complex C1, C2 , C3 ; C1 =
        complex(2 .5, 3.5) ; C2 =
        complex(1.6, 2.7); C3 = C1 +
        C2;
        cout << "C1 =" ;
        C1.display(); cout <<
        "C2 = "; C2.display();
        cout << "C3=";
        C3.display();
        }

```

The output of Program would be:

```

C1 = 2.5 +
3.5i C2 = 1.6
+ 2.7i C3 =
4.1 + 6.2i

```

Let us have a close look at the function operator+() and see how the operator overloading is implemented.

```

complex complex :: operator+(complex c)
{
        complex temp;
        temp.x = x + c.x;
        temp.y = y + c.y;
        return(temp);
}

```

We should note the following features of this function:

1. It receives only one complex type argument explicitly.
2. It returns a complex type value.
3. It is a member function of complex.

The function is expected to add two complex values and return a complex value as the result but receives only one value as argument. Where does the other value come from? Now let us look at the statement that invokes this function:

```
C3=C1 + C2;           //invokes operator+() function
```

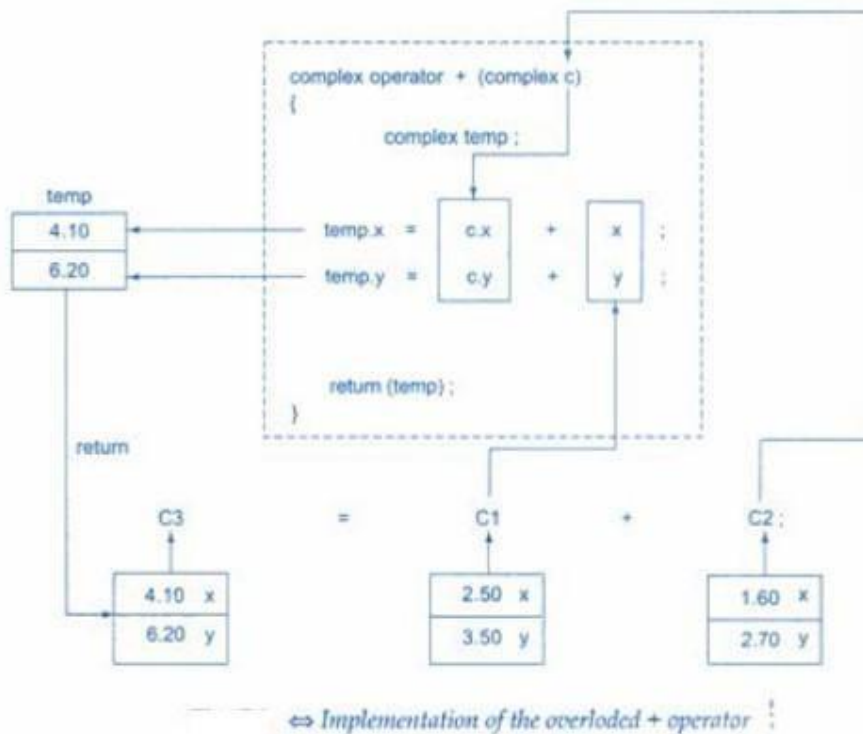
We know that a member function can be invoked only by an object of the same class. Here, the **object C1(Left Hand Operand must be an object)** takes the responsibility of invoking the function and C2 plays the role of an argument that is passed to the function. The above invocation statement is equivalent to,

```
C3 =C1.operator+(C2) : // usual function call syntax
```

Therefore, in the operator+() function, the data members of C1 are accessed directly and the data members of C2 (that is passed as an argument) are accessed using the dot operator. Thus both the objects are available for the function. For example in the statement,

```
temp.x = x + c.x;
```

c.x refers to the object C2 and x refers to the object C1. temp.x is the real part of temp that has been created specially to hold the results of addition of C1 and C2. The function returns the complex temp to be assigned to C3.



OVERLOADING BINARY OPERATORS USING FRIENDS

Friend function also used to overload binary operators in the place of member functions, the only difference being that a friend function requires two arguments to be explicitly passed to it, while a member function requires only one.

The complex number program discussed in the previous section can be modified using a friend operator function as follows:

1. Replace the member function declaration by the friend function declaration. `friend complex operator + (complex, complex);`
2. Redefine the operator function as follows: `complex operator+(complex c1, complex c2)`
{
complex c; c.num1=c1.num1+c2.num1; c.num2=c1.num2+c2.num2; return (c);
}

In this case, the statement `sum=C1 + C2;` is equivalent to `sum =operator+(C1, C2);`

In most cases, we will get the same results by the use of either a friend function or a member function. Why then an alternative is made available? There are certain situations where we would like to use a friend function rather than a member function. For instance, consider a situation where we need to use two different types of operands for a binary operator, say, one an object and another a built-in type data as shown below,

`A=B + 2;` (or `A=B * 2;`) where A and B are objects of same class. This will work for a member function but the statement,

`A = 2 + B ;` (or `A =2 * B`)

will not work. This is because the left-hand operand which is responsible for invoking the member function should be an object of the same class. However friend function allows both approaches. How?

It may be recalled that an object need not be used to invoke a friend function but can be passed as an argument.. Thus, we can use a friend function with a built- in type data as the *left-hand* operand and an object as the *right-hand* operand.

```
#include<iostream.h>
class Complex
{
int num1, num2; public:
void accept()
{
cout<<"\n Enter Two Complex Numbers : ";
cin>>num1>>num2;
}
//Overloading '+' operator using Friend function
friend Complex operator+(Complex c1, Complex c2);
```

```

void display()
{
cout<<num1<<"+"<<num2<<"i"<<"\n";
}
};
Complex operator+(Complex c1, Complex c2)

{
Complex                c;
c.num1=c1.num1+c2.num1;
c.num2=c1.num2+c2.num2; return(c);
}
int main()
{
Complex c1,c2, sum;      //Created Object of Class Complex i.e c1 and c2

c1.accept(); //Accepting the values
c2.accept();
sum = c1+c2; //Addition of object
cout<<"\n Entered Values : \n";
cout<<"\t";
c1.display();    //Displaying user input
values cout<<"\t";
c2.display();
cout<<"\n Addition of Real and Imaginary Numbers : \n";
cout<<"\t";
sum.display(); //Displaying the addition of real and imaginary numbers
return 0;
}

```

MANIPULATION OF STRINGS USING OPERATORS

ANSI C implements strings using character arrays, pointers and string functions. There are no operators for manipulating the strings. One of the main drawbacks of string manipulations in C is that whenever a string is to be copied the programmer must first determine its length and allocate the required amount of memory. Although these limitations exist in C++ as well, it permits us to create our own definitions of operators that can be used to manipulate the strings very much similar to the decimal numbers. (Recently, ANSI C++ committee has added a new class called string to the C++ class library that supports all kinds of string manipulations.)

For example we shall be able to use statements

like `strng3 = string1 + string2;`

`if(string1 >=string2) string = string1 ;`

Strings can be defined as class objects which can be then manipulated like the built-in types.

Example program:

```
#include<iostream>
```

```
#include<string.h>
```

```
class String
```



```

{
char str[20]; public:
void accept_string()
{
    cout<<"\n Enter String    ";
    cin>>str;

}
void display_string()
{
    cout<<str;
}
String operator+(String x) //Concatenating String
{

String s; strcat(str,x.str);
strcpy(s.str,str);
return s;
}
};

int main()
{
    String  str1,  str2,
str3;  str1.accept_string();
str2.accept_string();
cout<<"\n ----- ";
cout<<"\n\n First String is :  "; str1.display_string();
//Displaying First String cout<<"\n\n Second String
is :  "; str2.display_string(); //Displaying  Second
String cout<<"\n ----- ";
str3=str1+str2;          //String is concatenated. Overloaded '+'
operator cout<<"\n\n Concatenated String is: ";
str3.display_string();
return 0;
}

```

RULES FOR OVERLOADING OPERATORS

Although it looks simple to redefine the operators, there are certain restrictions and limitations in overloading them. Some of them are listed below:

- Only existing operators can be overloaded. New operators cannot be created.
- The overloaded operator must have at least one operand that is of user-defined type.
- Overloaded operators follow the syntax rules of the original operators. They cannot be overridden.
- There are some operators that cannot be overloaded. (See Table 1.)
- We cannot use friend functions to overload certain operators. (See Table 2.)

- However member functions can be used to overload them.
- Unary operators, overloaded by means of a member function, take no explicit arguments and return no explicit values, but, those overloaded by means of a friend function, take one reference argument (the object of the relevant class).
- Binary operators overloaded through a member function take one explicit argument and those which are overloaded through a friend function take two explicit arguments.
- When using binary operators overloaded through a member function, the left hand operand must be an object of the relevant class.

Table 1

Operators that cannot be overloaded are,

Sizeof	Size of operator
---------------	-------------------------

- Membership operator
- ? : Conditional operator
- . * Pointer-to-member operator
- : : Scope resolution operator

Table 2

Where a friend cannot be used,

- = Assignment operator
- () Function call operator
- [] Subscripting operator
- > Class member access operator

Multiple Choice questions.

1. What is the keyword used in operator overloading to create a operator function
 - A. Class
 - B. Destructor
 - C. Constructor
 - D. Operator**
2. What is the role of a constructor in classes?
 - a) To modify the data whenever required
 - b) To destroy an object
 - c) To initialize the data members of an object when it is created**
 - d) To call private functions from the outer world
3. What is a copy constructor?
 - a) A constructor that allows a user to move data from one object to another
 - b) A constructor to initialize an object with the values of another object**
 - c) A constructor to check the whether to objects are equal or not
 - d) A constructor to kill other copies of a given object.
4. How constructors are different from other member functions of the class?
 - a) Constructor has the same name as the class itself
 - b) Constructors do not return anything
 - c) Constructors are automatically called when an object is created
 - d) All of the mentioned**
5. How many types of constructors are there in C++?
 - a) 1
 - b) 2
 - c) 3**
 - d) 4
6. Which of the following is the set of operators that can't be overloaded in C++?
 - A. +=, ?, ::, >>
 - B. >>, <<, ?, *, sizeof()
 - C. ::, ., .*, ?:, sizeof()**
 - D. ::, ->, *, new, delete
7. How many argument/s is required ,when overloading unary operators using Friend function.
A. Zero B. One C. Two D. Three.
8. What is a symbol preceding a destructor
 - A. ! B. ? **C. ~** D. *
9. How many argument/s is required ,while overloading binary operators using member function(L1)
A. Zero B. One C. Two D. Three
10. The process of making an operator to exhibit different behaviours in different instances is known as

A. Operator overloading

- B.** Function overloading
- C.** Dynamic binding
- D.** Late Binding

11. Which one of the following doesn't accept any parameter

- A.** Default Constructor
- B.** Parameterized Constructor
- C.** Copy Constructor
- D.** Static Constructor

12. How many default constructors per class are possible?

B) Only one

- C)** Two
- D)** Three
- E)** Unlimited

13. Using friend operator function, which of following perfect set of operators may not be overloaded.

- A.** = , () , [] , ->
- B.** << , == , [] , >>
- C.** ? , = , () , ++
- D.** += , ? , :: , >>

14. Can you identify which of the below statement is true about Constructors

- a. construct the data members
- b. Both initialize the objects & create new data members
- c. Constructs the data type for objects
- d. initialize the objects**

15. can you judge on which of the following statement is correct?

- a. A constructor is called at the time of declaration of an object.**
- b. A constructor is called at the time of use of an object.
- c. A constructor is called at the time of declaration of a class.
- d. A constructor is called at the time of use of a class.

16. Which of the following statement is correct about the constructors and destructors?

- a. Destructors can take arguments but constructors cannot.
- b. Constructors can take arguments but destructors cannot.**
- c. Destructors can be overloaded but constructors cannot be overloaded.
- d. Constructors and destructors can both return a value

17. Can you judge which of the below can be used to identify the copy constructor of class type X?

- A)** (X&) **B) X(&X)** **C)** X(X&)
- D)** X(X)

18. Which of the below is supported for Constructors declaration.
- Constructors should be declared in the Private section
 - Constructors should be declared in the Public section**
 - Constructors should be declared in the protected section.
 - Constructors should be declared as Static
19. Can you judge which of the below can be said about constructors, “when more than one constructor function is defined in a class”
- A. Parameterized B. Copied **C. Overloaded** D. Inherited
20. Can you Identify the correct method for a copy constructor to receive its arguments
- either pass-by-value or pass-by-reference
 - only pass-by-value
 - only pass-by-reference**
 - only pass by address
21. Can you judge on which of following statement should be taken into consideration, In case of binary operator overloading with member function,
- Right hand operand must be object.
 - Left hand operand must be object.**
 - Both the operands must be objects.
 - All of these should be considered.
22. What is the role of destructors in Classes?
- To modify the data whenever required
 - To destroy an object when the lifetime of an object ends
 - To initialize the data members of an object when it is created
 - To call private functions from the outer world
23. Can you assess the following statements and identify the statements which are Not True about destructor?
- It is invoked when object goes out of the scope
 - like constructor, it can also have parameters
 - It can be virtual
 - It can be declared in private section
 - It bears same name as that of the class and precedes Lambda sign.
- A. Only 2, 3, 5 B. Only 2, 3, 4 **C. Only 2, 4, 5** D. Only 3, 4, 5
24. Assume class TEST. Identify which of the following statements is/are responsible to invoke copy constructor?
- A. TEST T2(T1) B. TEST T4 = T1 C. T2 = T1 **D. both A and B**
25. Examine the below and identify the correct form used to develop operator function
- return_type classname :: operator op (arglist)**
{
Function body
}
 - return_type operator :: classname op (arglist)
{
Function body

```
}  
c. classname :: operator op (arglist)  
{  
Function body  
}  
d. return_type classname :: op (arglist)  
{  
Function body  
}
```

Long Answer Questions

1. What is the purpose/need for using a Constructor? List the special characteristics of the Constructor.
 2. Explain default constructor with syntax and example program.
 3. Explain parameterized constructor with syntax and example program.
 4. Explain the features of copy constructor with an example program.
 5. Explain Destructors with syntax and example.
 6. Explain multiple constructors in a class with example.
-
1. Survey on operator overloading and rules for overloading operators
 2. Analyze constructors with default arguments with an example program?
 3. Analyze overloading of Unary operators with an example program.
 4. Analyze overloading of Binary operators using member function with an example program.
 5. Analyze overloading of Binary operators using friend function with an example program.
 6. Explain manipulation of strings using operators

UNIT-III
FUNCTION OVERLOADING AND INHERITANCE
CHAPTER 5: FUNCTION OVERLOADING

INTRODUCTION

We know that functions play an important role in C program development. Dividing a program into functions is one of the major principles of top down, structured programming. Another advantage of using functions is that it is possible to reduce the size of a program by calling and using them at different places in the program.

Recall that we have used a syntax similar to the following in developing C

```
programs. void show();           //function declaration
main()
{
.....
show();                          // function call
.....
}
void show()                      //function definition
{
.....
.....                          //function body
.....
}
```

When the function is called, control is transferred to the first statement in the function body.

The other statements in the function body are then executed and control returns to the main program when the closing brace is encountered. C++ is no exception. Functions continue to be the building blocks of C++ programs. In fact, C++ has added many new features to functions to make them more reliable and flexible. Like C++ operators, a C++ function can be overloaded to make it perform different tasks depending on the arguments passed to it.

FUNCTION PROTOTYPING AND FUNCTION DEFINITION

Function *prototyping* is one of the major improvements added to C++ functions. The prototype describes the function interface to the compiler by giving details such as the number and type of arguments and the type of return values. With function prototyping, a *template* is always used when declaring and defining a function. When a function is called, the compiler uses the template to ensure that proper arguments are passed, and the return value is treated correctly. Any violation in matching the arguments or the return types will be caught by the compiler at the time of compilation itself. These checks and controls did not

exist in the conventional C functions.

It was introduced first in C++ by Stroustrup and the success of this feature inspired the ANSI C committee to adopt it. However, there is a major difference in prototyping between C and C++. While C++ makes the prototyping essential, ANSI C makes it optional, perhaps, to preserve the compatibility with classic C.

Function prototype is a declaration statement in the calling program and is of the following form;

Return_type function_name(argument-list); // function declaration

float volume(int x, float y, float z); or
float volume(int, float, float); is acceptable.

Float volume(int x, float y, z); is illegal.

In the **function definition**, names are required because the arguments must be referenced inside the function. Example:

```
float volume(int a, float b, float c)
{
float v=a*b*c;
.....
.....
}
```

The function volume() can be invoked in a program as follows:

```
float cube1= volume(b1 ,w1 , h1); // Function call
```

The variable b1, w1, and h1 are known as the actual parameters which specify the dimensions of cube1. Their types (which have been declared earlier) should match with the types declared in the prototype. Remember, the calling statement should not include type names in the - argument list.

We can also declare a function with an *empty argument list*, as in the following example: void display();

In C++, this means that the function does not pass any parameters. It is identical to the statement void display(void);

INTRODUCTION TO FUNCTION OVERLOADING

Function overloading means two or more functions having same name, but differ in the number of arguments or data type of arguments. Therefore it is said that (function name) is overloaded. Function overloading therefore is the process of defining same function name to carry out similar types of activities with various data items. The advantages of function overloading are:

- When different functions are created for different operations, then user has to call respective function depending on the situation. Instead, for different situations if the same function is called with different arguments using function overloading, then the compiler automatically decides about the appropriate function by comparing the argument types used in the call to the function and calls the required function. Thus the code is executed faster.
- It is easier to understand the flow of information and debug.
- Code maintenance is easy.
- Easier interface between programs and real world objects.

DEFINITION AND DECLARATION OF OVERLOADED FUNCTIONS

The main factor in function overloading is a function's argument list. C++ can distinguish overloaded functions by the number and type of arguments. If there are two functions having same name and different types of arguments or different number of arguments, then function overloading is invoked automatically by the compiler. **Function Overloading is also known as Compile time polymorphism.**

Example 1:

```
int test() { }  
int test(int a) { }  
float test(double a) { }  
int test(int a, double b) { }
```

Here, all 4 functions are overloaded functions because argument(s) passed to these functions are different.

Notice that, the return type of all these 4 functions is not same. Overloaded functions may or may not have different return type but it should have different argument(s).

// Error code

```
int test(int a) { }  
double test(int b){  
}
```

The number and type of arguments passed to these two functions are same even though the return type is different. Hence, the compiler will throw error.

```
int sum(int a, int b);  
float sum(float p, float q);
```

The function sum() that takes two integer arguments is different from the function sum() that takes two float arguments. This is function overloading. To overload a function, each overloaded function must be declared and defined separately.

Example 3:

```
int product(int p, int q, int r);  
float product(float x, float y, float  
z); int product(int p, int q, int r)  
{  
cout<<"product="<<p*q*r<<endl;  
}
```

```
float product(float x, float y, float z)
{
cout<<product=" "<<x*y *z<<endl;
}
```

In this example the function product() is overloaded two times, first time with three integer values and integer as return value, second time with three float values and return type being a float. The compiler automatically chooses the right type of function depending on the number of arguments.

DIFFERENT WAYS TO OVERLOAD A FUNCTION

1. By changing number of Arguments.
2. By having different types of argument.

FUNCTION OVERLOADING: DIFFERENT NUMBER OF ARGUMENTS

In this type of function overloading we define two functions with same names but different number of parameters of the same type. For example, in the below mentioned program we have made two sum() functions to return sum of two and three integers.

```
int sum (int x, int y)
{
cout << x+y;
}
// second overloaded
defintion int sum(int x, int y,int z)
{
cout << x+y+z;
}
```

Here sum() function is said to overloaded, as it has two definition, one which accepts two arguments and another which accepts three arguments. Which sum() function will be called, depends on the number of arguments.

```
int main()
{
sum (10, 20);          // sum() with 2 parameter will be called
sum(10, 20, 30);       //sum() with 3 parameter will be
called
}
```

FUNCTION OVERLOADING: DIFFERENT TYPE OF ARGUMENTS

In this type of overloading we define two or more functions with same name and same number of parameters, but the type of parameter is different. For example in this program, we have two sum() function, first one gets two integer arguments and second one gets two double arguments.

```
int sum(int x, int y)          // first definition
{
cout<< x+y;
}
```

```
double sum(double x, double y) // second overloaded definition
{
cout << x+y;
}
int main()
{
sum      (10,20);
sum(10.5,20.5);
}
```

The following programming example shows how overloaded functions can be called.

Program: To compute volume of cube and cylinder and cuboid using overloaded functions. #include<iostream.h>

```
#include<conio.h
> const float
pi=3.14; class
funoverload
{
public:
int volume(int l)          // Volume of Cube
{
return l*l*l;
}
float volume(float r, float h) // Volume of cylinder
{
return (pi*r*r*h);
}
double volume(float l, float b, float h)// Volume of cuboid
{
return (l*b*h);
}
};
int main()
{
clrscr();
funoverload f1;
cout<<"Volume of the Cube: "<<f1.volume(10)<<endl;
cout<<"Volume of the Cylinder: "<<f1.volume(2.0,3.0)<<endl;
cout<<"Volume of the Cuboid: "<<f1.volume(5.0,6.0,7.0)<<endl;
return 0;
getch();
}
```

Volume of the Cube: 1000

Volume of the Cylinder:

37.68 Volume of the Cuboid:

210

The above program finds volume of cube, cuboid, and cylinder depending on the number and type of arguments provided as input to the function called volume. Therefore though all functions have the same name volume, appropriate function is selected based on data type of argument list. The compiler selects the required function through function overloading.

CHAPTER-6: INHERITANCE: EXTENDING CLASSES

INTRODUCTION

Reusability is yet another important feature of OOP. It is always nice if we could reuse something that already exists rather than trying to create the same all over again. It would not only save time and money but also reduce frustration and increase reliability. For instance, the reuse of a class that has already been tested, debugged and used many times can save us the effort of developing and testing the same again.

Fortunately, C++ strongly supports the concept of *reusability*. The C++- clB88es can be reused in several ways. Once a class has been written and tested, it can be adapted by other programmers to suit their requirements. This is basically done by creating new classes, reusing the properties of the existing ones. The mechanism of deriving a new class from an old one is called *inheritance (or derivation)*. The old class is referred to as the *base class* and the new one is called the *derived class or subclass*.

The derived class inherits 110me or all of the traits from the base class. A class can also inherit properties from more than one class or from more than one level. A derived class with only one base class, is called *single inheritance* and one with several base classes is called *multiple inheritance*. On the other hand, the trait a: of one class may be inherited by more than one class. This process is known as *hierarchical inheritance*. The mechanism of deriving a class from another 'derived class' is known as *multilevel inheritance*.

Definition:- Inheritance is the capability of one class to inherit properties from another class.

Base Class: It is the class whose properties are inherited by another class. It is also called Super Class.

Derived Class: It is the class that inherits properties from base class or classes. It is also called Sub Class.

Inheritance has following advantages:

1. Reusing existing code
2. Faster development time
3. Easy to maintain
4. Easy to extend
5. Memory utilization

DEFINING DERIVED CLASSES

When you declare a class, you can indicate what class it derives from by writing a colon after the class name, the type of derivation (public or private or protected) and the class from which it derives.

```
class derived_class_name : visibility_mode base_class_name
{
// .....
//members of the derived class
};
```

The colon (:) indicates that the derived-class-name is derived from the base-class-name.

The visibility-mode specifies the type of derivation. **If no visibility mode is specified, then by default the visibility mode is considered private.** Visibility mode specifies whether the features of the base class are privately derived or publicly derived.

The body of the derived class contains its own data members and member function. Like base class definition, the derived class definition must be terminated by a semicolon. Note that all members of the class except private are inherited. Following are some examples of derived class definition.

```
class father // Base class
```

```
{
```

```
private:
```

```
char name[50];int age;
```

```
public:
```

```
char caste[50];int boys;
```

```
int girls;
```

```
void readdata();
```

```
void printdata();
```

```
};
```

```
class son: public father    //public derived class
```

```
{
```

```
private:
```

```
char company[50];float salary;
```

```
public:
```

```
void getdata(); void
```

```
display();
```

```
};
```

VISIBILITY MODE

The visibility mode (private, public and protected) in the definition of the derived class specifies whether features of the base class are privately derived or publicly derived or protectedly derived. The visibility mode basically controls the access specifier to be for inheritable member of base class in the derived class.

The role of visibility modes:

Table showing all the Visibility Modes

Base class	Derived Class		
	Public mode	Private Mode	Protected Mode
Private	Not Inherited	Not Inherited	Not Inherited
Protected	Protected	Private	Protected
Public	Public	Private	Protected

Public Inheritance (mode)

This is the most used inheritance mode. In this

- The public members of a base class become public members of the derived class.
- The private members of a base class cannot be inherited to the derived class.
- The protected members of a base class stay protected in a derived class.

class subclass: **public** baseclass

```
class student //base class
```

```
{
```

```
private:
```

```
int rollno;
```

```
char name[50];float per;
```

```
public:
```

```
void input(); void
```

```
output();
```

```
};
```

```
class comp: public student //publicly derived class
```

```
{
```

```
private:
```

```
int marks;public:
```

```
void read();
```

```
void write();
```

```
};
```

Private Inheritance

- The public members of a base class become the private members of the derived class.
- The private members of a base class cannot be inherited to the derived class.
- The protected members of a base class become private in a derived class.

Protected Inheritance

- The public members of a base class become protected in a derived class.
- The private members of a base class cannot be inherited to the derived class.
- The protected members of a base class stay protected in a derived class.

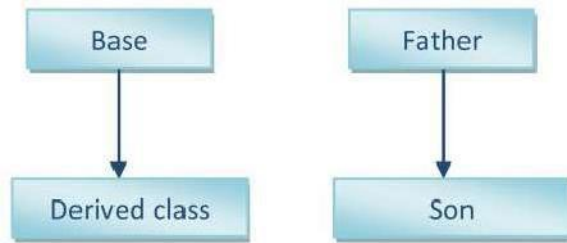
- class subclass : **protected** Baseclass

DIFFERENT TYPES OF INHERITANCE

1. Single Inheritance
2. Multiple Inheritance
3. Multilevel Inheritance
4. Hierarchical Inheritance
5. Hybrid Inheritance.

SINGLE INHERITANCE

If a class is derived from a single base class, it is called as single inheritance. In other words a derived class with only one baseclass, is called *single inheritance*.



Example:

```
#include<iostream
.h> #include<conio.h
> class rectangle
{
protected: public:
float length, breadth,area; void accept()
{
cout<<"Enter the length:";cin>>length;
cout<<"Enter the breadth:";cin>>breadth;
}
void calculate_area()
{
area=length*breadth;
cout<<"The area of rectangle:"<<area<<endl;
}
};
class cuboid : public rectangle
{
float height, volume;
public:
void getdata()
{
accept(); cout<<"Enter the height:";
cin>>height;
}
void calculate_volume()
{
volume=length*breadth*height;
cout<<"The volume of cuboid is:"<<volume<<endl;
}
};
void main()
{
cuboidc; clrscr()
;
c.getdata(); c.calculate_area(); c.calculate_volume(
); getch();
}
```

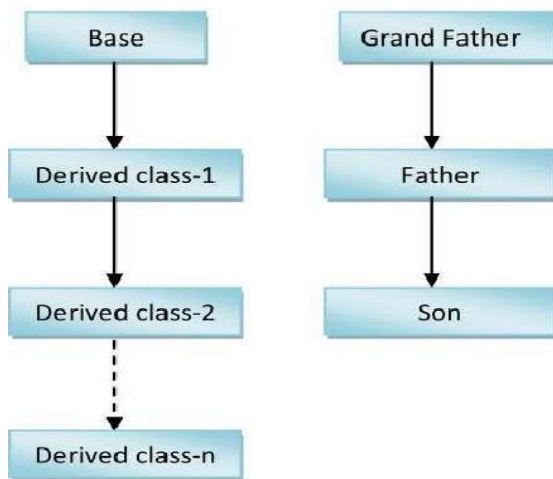
Output:

Enter Length: 5
Enter breadth :4
Enter height: 5
The area of rectangle: 20
The volume of cuboid is: 100

Here class rectangle is the base class and cuboid is the derived class. Class cuboid inherits the properties length and breadth from base class and it has its own property height.

MULTILEVEL INHERITANCE

The mechanism of deriving a class from another 'derived class' is known as ***multilevel inheritance***. The classes can also be derived from the classes that are already derived. This type of inheritance is called multilevel inheritance.



The class A serves as a base class for the derived class B, which in turn serves as a base class for the derived class C. The class B is known as intermediate base class since it provides a link for the inheritance between A and C. The chain ABC is known as inheritance path.

A derived class with multilevel inheritance is declared as follows:

```
class A{ ... } ; // Base class
class B: public A { ... } ; // B derived from A
class C: public B { ..... } ; // C derived from B
```

This process can be extended to any number of levels.

Let us consider a simple example. Assume that the test results of a batch of students are stored in three different classes. Class student stores the roll number, class test stores the marks obtained in two subjects and class result contains the total marks obtained in the test. The class result can inherit the details of the marks obtained in the test and the roll number of students through multilevel inheritance.

Example:

```
#include <iostream.h>
class student
{
protected: int roll_number; public: void
getnumber(int);
void putnumber( );
};
void student : :getnumber(int a)
{
Roll_number= a;
}
void student :: putnumber()
{
cout << "Roll number="<<roll_number;
}
class test : public student
{
protected:
float sub1;float sub2;
public:
void get_marks(float, float);
void put_marks( );
};
void test :: get_marks(float x, float y)
{
sub1= x;
sub2 = y;
}
void test : : put_marks()
{
cout << "Marks in Sub1=" << sub1<< "\n" ; cout
<< "Marks in Sub2 =" << sub2
<< "\n";
}
class result : public test
{
float total;public:
void display( );
};
void result :: display( )
{
total = sub1+sub2;
putnunber();
put_marks();
cout <<"Total ="<<total ;
}
void main()
{
result student1; // student1 created
```

```

student1.getnumber(101);
student1.get_marks(75.0,59.5);
student1.display();
}

```

Program displays the following output: Roll

Humber: 101

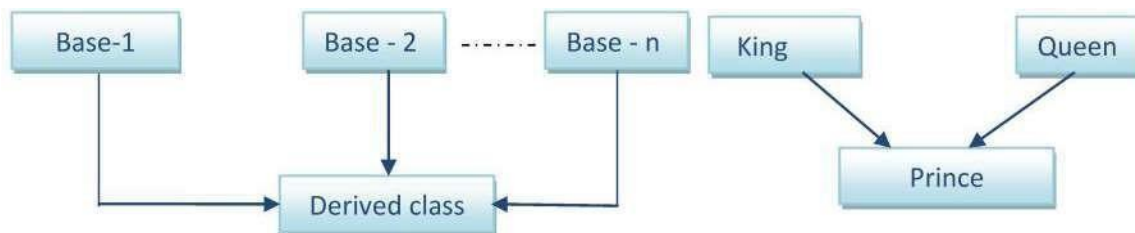
Marks in Sub1 = 75

Marks in Sub2 = 59.5

Total=134.5

MULTIPLE INHERITANCE

If a class is derived from more than one base class, it is known as multiple inheritance. Multiple inheritance allows us to combine the features of several existing classes as a starting point for defining new classes. It is like a child inheriting the physical features of one parent and the intelligence of another.



The syntax of n derived class with multiple base classes is follows:

```

class D: visibility B-1, visibility B-2 ...
{
    .....
    .....(Body of D)
    .....
};

```

where, visibility may be either public or private. The base classes are separated by commas.

```
#include<iostream.h>
```

```
class M
```

```
{
```

```
protected:      int m;
```

```
public:
```

```
void ge_ m(int);
```

```
};
```

```
class N
```

```
{
```

```
protected:
```

```
int    n;
```

```
public:
```

```
void get_n(int);
```

```
class P : public M, public N
{
public:
void display(void);
};

void M :: get_m(int x)
{
m =x;
}

void N :: get_n(int y)
{
n =y;
}

void P :: display()
{
cout << "m=" << m<< "\n";
cout << "n=" << n << "\n";
cout << "m*n=" << "m*n" << "\n";
}

void main( )
{
P p;
p .get_m(10) ;
p.get_n(20) ;
p.display() ;
}
```

The output of Program would be:

```
m =10
n = 20
m*n = 200
```

6.3.4.1 Ambiguity Resolution in Inheritance

Occasionally, we may face a problem in using the multiple inheritance, when a function with the same name appears in more than one base class. Consider the following two classes.

```
class M
{
public:
void display(void)
{
cout<<"ClassM";
};
}

class N
{
public:
void display(void)
{
cout << "Class N\n";
}
};
```

Which display() function is used by the derived class when we inherit these two classes? We can solve this problem by defining a named instance within the derived class, using the classresolution operator with the function as shown below:

In this case, the function in the derived class overrides the inherited function and therefore, a simple call to display() by B type object will invoke function defined in B only. However, we may invoke the function defined in A by using the scope resolutionoperator to specify the class. Example:

```
void main()
{
    B      b;
    b.display();
    b.A:
    :display();
    b.B: :display();
}
```

This will produce the following output:

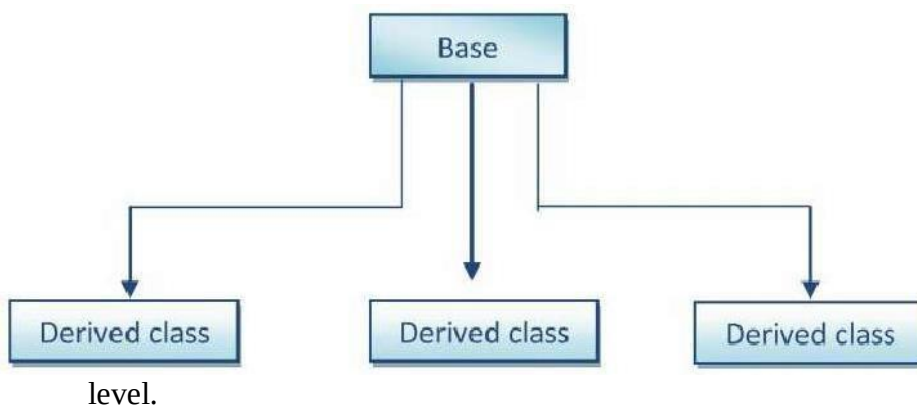
B

A

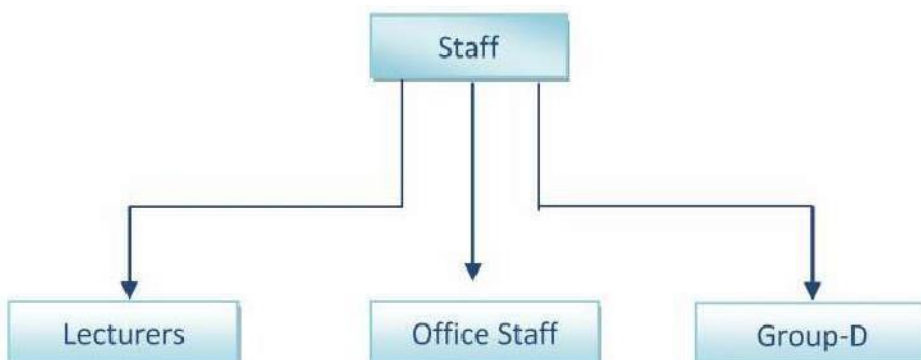
B

HIERARCHICAL INHERITANCE

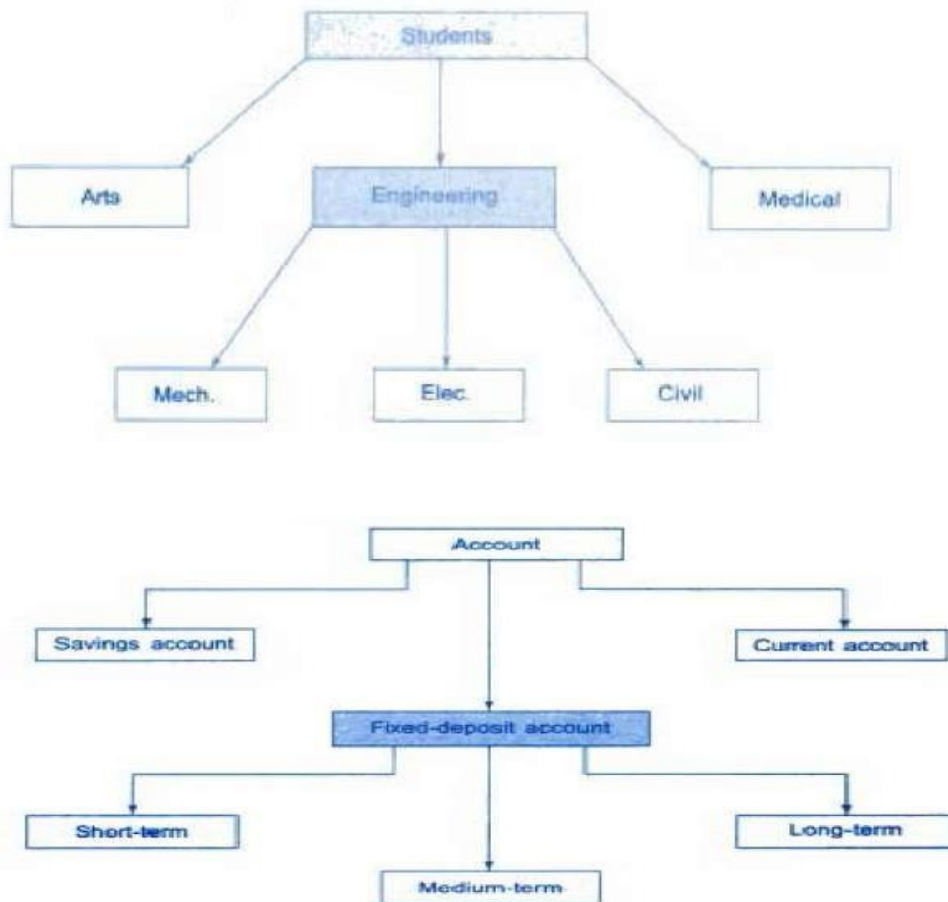
Another interesting application of inheritance is to use it as a support to the hierarchical design of a program. Many programming problems can be cast into a hierarchy where certain features of one level are shared by many others below that



Example for Hierarchical Inheritance:



As an example, below figure shows a hierarchical classification of students in a university. Another example could be the classification of accounts in a commercial bank as shown in below figure. All the students have certain things in common and, similarly, all the accounts possess certain common features. If a number of classes are derived from a single base class, it is called as hierarchical inheritance.



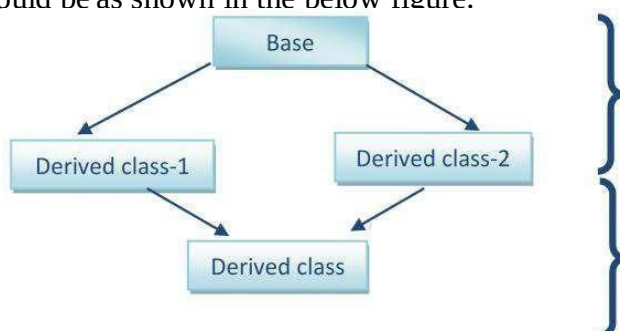
In C++, such problems can be easily converted into class hierarchies. The base class will include all the features that are common to the subclasses. A subclass can be constructed by inheriting the properties of the base class. A subclass can serve as a base class for the lower level classes and so on.

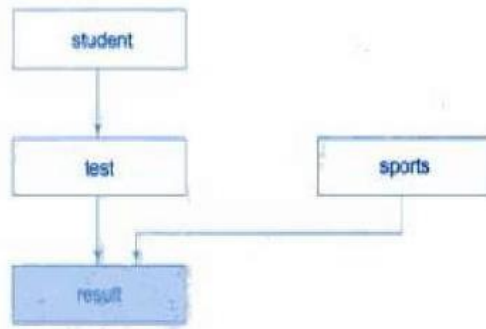
HYBRID INHERITANCE

Hybrid Inheritance is combination of Hierarchical and Multiple Inheritance. Or simply, hybrid

inheritance is a combination of more than one types of inheritance.

There could be situations where we need to apply two or more types of inheritance to design a program. For instance, consider the case of processing the student result discussed in hierarchical inheritance. Assume that we have to give weightage for sports before finalizing the results. The weightage for sports is stored in a separate class called sports. The new inheritance relationship between the various classes would be as shown in the below figure.



**Example:**

```

#include
<iostream.h>    class
student
{
protected: int  roll_number;
public: void getnumber(int);
void putnumber( );
};
void student : :getnumber(int a)
{
roll_number= a;
}
void student :: putnumber()
{
cout << "Roll number="<<roll_number;
}
class test : public student
{
protected:
float sub1;float sub2;
void get_marks(float,
float); void put_marks( );
public:
void test :: get_marks(float x, float y)
{
sub1= x;
sub2 = y;
}
void test : : put_marks()
{
cout << "Marks in Sub1=" << sub1<< "\n"
; cout << "Marks in Sub2 =" << sub2
<< "\n";
}
class sports
{
protected: float score;
public: void getscore(float s)
{
score=s;
}
}
  
```

```
}  
void putscore()  
{  
cout<<"Sports score="<<score;  
}  
};  
class result : public test, public sports  
{  
float total;  
public:  
void display( );  
};  
void result :: display( )  
{  
    total =  
    sub1+sub2  
    +score;  
    putnumber(  
    );  
    put_marks(  
    );  
    putscore();  
    cout <<"Total ="<<total ;  
}  
void main()  
{  
result student1; // student1  
created  
student1.getnumber(101);  
student1.get_marks(75.0, 59.5);  
student1.getscore(7.0);  
student1.display();  
}
```

Output:

Roll

no:

101

Mark

s

Obtai

ned:

Sub1=75.0

Sub2=59.5

Sports score: 7.0

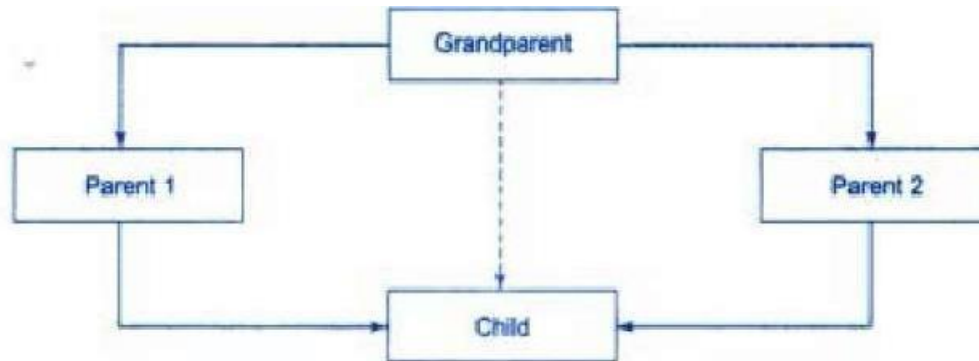
Total Score: 141.5

VIRTUAL BASE CLASS

We have just discussed a situation which would require the use of both the multiple and multilevel inheritance. Consider a situation where all the three kinds of inheritance, namely, multilevel, multiple and hierarchical inheritance, are involved. This is illustrated in below figure. The 'child' has two direct base classes 'parent1' and 'parent2' which themselves have a common base class 'grandparent'. The 'child'

inherits the properties of 'grandparent' via two separate paths. It can also inherit directly as shown by the broken line. The 'grandparent' is sometimes referred to as indirect base class.

Inheritance by the 'child' as shown in below figure might pose some problems. All the public and protected members of 'grandparent' are inherited into 'child' twice, first via 'parent1' and again via 'parent2'. This means, 'child' would have duplicate sets of the members inherited from 'grandparent'. This introduces ambiguity and should be avoided.



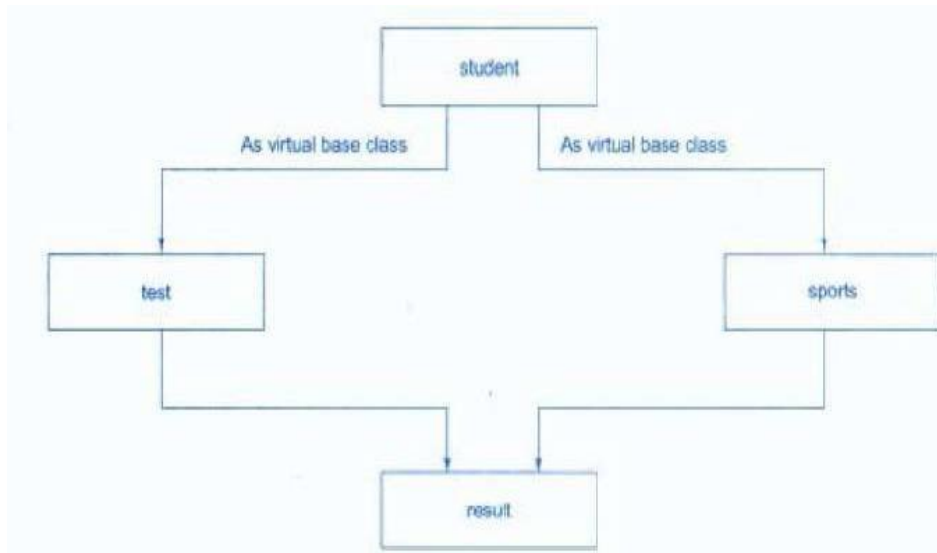
The duplication of inherited members due to these multiple paths can be avoided by making the common base class (ancestor class) as *virtual base class* while declaring the **direct or intermediate base classes as shown below.**

```

class A                                // grandparent
{
    .....
    .....
};
class B1 : virtual public A            // parent1
{
    .....
    .....
};
class B2 : public virtual A           // parent2
{
    .....
    .....
};
class C : public B1, public B2        //child
{
    .....                          // only one copy of A
    .....                          // will be Inherited
};
  
```

When a class is made a virtual base class, C++ takes necessary care to see that only one copy of that class is inherited, regardless of how many inheritance paths exist between the virtualbase class and a derived class.

For example, consider the student results processing system. Assume that the class sports derives the roll_number from the class student. Then, the inheritance relationship will be as shown in below figure.

**Example:**

```

#include
<iostream.h>
class student
{
    protected:      int
    roll_number;
    public:          void
    getnumber(int);
    void putnumber( );
};
void student : :getnumber(int a)
{
    roll_number= a;
}
void student :: putnumber()
{
    cout << "Roll number="<<roll_number;
}
class test : virtual public student
{
    protected:
    float sub1;float sub2;
    void get_marks(float,
    float); void put_marks( );
    public:
    void test :: get_marks(float x, float y)
    {
        sub1= x;
        sub2 = y;
    }
    void test : : put_marks()
    {
        cout << "Marks in Sub1=" <<
        sub1<< "\n" ; cout << "Marks in
        Sub2 =" << sub2 << "\n";
    }
}
class sports: public virtual student
{

```

```
protected: float score;
public: void getscore(float s)
{
    score=s;
}
void putscore()
{
    cout<<"Sports score="<<score;
}
};
class result : public test, public sports
{
    float total;public:
    void display( );
};
void result :: display( )
{
    total =
    sub1+sub2+sco
    re;  putnnumber();
    put_marks();
    putscore();
    cout<<"total is:"<<total;
}
void main()
{
    result    student1;    //
    student1    created
    student1.getnumber(102);
    student1.get_marks(30.5,
    25.5);
    student1.getscore(7.0);
    student1.display();
}
```

Output:

Roll no:

102

Marks Obtained:

Sub1=30.5

Sub2=25.5

Sports score: 7.0

Total Score: 63

In the above example, only one copy of roll_number variable is inherited by the result class from the student class. Thus any changes made to roll_number by any of the derived classes are made to this single instance. This ensures data integrity.

ABSTRACT BASE CLASSES

An abstract class is one that is not used to create objects. An abstract class is designed only to act as a base class (to be inherited by other classes). It is a design concept in program development and provides a base upon which other classes may be built.

the class A is an abstract class since it is not used to create any objects. However, one should remember that an abstract class must have a pure virtual function.

The general form of using Abstract base class is

shown below:

```
class base          //abstract base class
{
    public: virtual void xyz()=0;      //pure virtual function
};
class derived : public base
{
    public: void xyz()
    {
        Cout<<"Abstract base class with pure virtual function";
    }
};
void main()
{
    base b;          //Error :no object for abstract class
    derived ob;
    ob.xyz();
}
```

Output:

Abstract base class with pure virtual function

CONSTRUCTORS IN DERIVED CLASSES

As we know, the constructors play an important role in initializing objects. One important thing to note here is that, as long as no base class constructor takes any arguments, the derived class need not have a constructor function.

However, if any base class contains a constructor with one or more arguments, then it is mandatory for the derived class to have a constructor and pass the arguments to the base class constructors. Remember, while applying inheritance we usually create objects using the derived class. Thus, it makes sense for the derived class to pass arguments to the base class constructor.

When both the derived and base classes contain constructors, the base constructor is executed first and then the constructor in the derived class is executed.

In case of multiple inheritances, the base classes are constructed in the order in which they appear in the declaration of the derived class. Similarly, in a multilevel inheritance, the constructors will be executed in the order of inheritance.

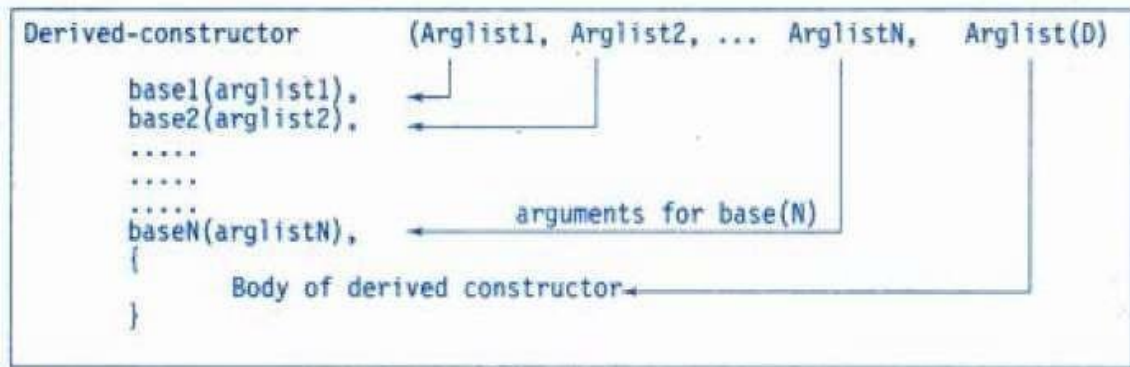
Since the derived class takes the responsibility of supplying initial values to its base classes, we supply the initial values that are required by all the classes together, when a derived class object is declared.

How are the value passed to the base class constructors so that they can do their

job?

C++ supports a special argument passing mechanism for such situations. The constructor of the derived class receives the entire list of values as its arguments and passes them on to the base constructors in the order in which they are declared in the derived class. The base constructors are called and executed before executing the statements in the body of the derived constructor.

The general form of defining a derived constructor is:



The header line of derived-constructor function contains two parts separated by a colon(:).

- The first part provides the declaration of the arguments that are passed to the derived-constructor and
- the second part lists the function calls to the base constructors.
base1(arglist1), base2(arglist2) ... are function calls to base constructors base1(),base2(), ... and therefore arglist1, arglist2, ... etc. represent the actual parameters that are passed to the base constructors.

Arglist1 through ArglistN are the argument declarations for base constructors base1 through baseN. ArglistD provides the parameters that are necessary to initialize the members of the derived class.

Example:

```

D(int a1 , int a2, float b1 , float b2, int d1) : A(a1, a2), B(b1, b2)// call to
constructor A first
then constructor B
{
    d = d1;           // executes its own body
}

```

A(a1, a2) invokes the base constructor A() and B(b1, b2) invokes another base constructor B().

The constructor D() supplies the values for these four arguments. In addition, it has one argument of its own. The constructor D() has a total of five arguments. D() may be invoked as follows:

```
D objD (5, 12, 2.5, 7.54, 30);
```

These values are assigned to various parameters by the constructor D() as

follows: 5 -> a1

12 -> a2

4 ->b2

30->d 1

The constructors for virtual base classes are invoked before any non-virtual baseclasses. If there are multiple virtual base classes, they are invoked in the order in which they are declared. Any non-virtual bases are then constructed before the derived class constructor is executed.

Table	Execution of base class constructors
Method of inheritance	Order of execution
<pre>Class B: public A { };</pre>	<pre>A() ; base constructor B() ; derived constructor</pre>
<pre>class A : public B, public C { };</pre>	<pre>B() ; base(first) C() ; base(second) A() ; derived</pre>
<pre>class A : public B, virtual public C { };</pre>	<pre>C() ; virtual base B() ; ordinary base A() ; derived</pre>

MEMBER CLASSES: NESTING OF CLASSES

Inheritance is the mechanism of deriving certain properties of one class into another. We have seen in detail how this is implemented using the concept of derived classes. C++ supports yet another way of inheriting properties of one class into another. This approach takes a view that an object can be a collection of many other objects. That is, a class can contain objects of other classes as its members as shown below:

```
class alpha { ..... };
class beta { ..... };
class gamma
{
alpha a;    // a is an object of alpha class
beta b;    // b is an object of beta class
};
```

all objects of gamma class will contain the objects a and b. This kind of relationship is called **containership or nesting**. Creation of an object that contains another object is very different than the creation of an independent object. A independent object is created by its constructor when it is declared with arguments. On the other hand, a nested object is created in two stages. First, the member objects are created using their respective constructors and then the other 'ordinary' members are created. This means, constructors of all the member objects should be called before its own constructor body is executed. This is accomplished using an initialization list in the constructor of the nested class.

```
class gamma
{
    alpha a; // a is object
    of alpha
    beta b; // b is
    object of beta
    public:
        gamma(arglist): a(arglist1), b(arglist2)
        {
            // constructor body
        }
};
```

arglist is the list of arguments that is to be supplied when a gamma object is defined. These parameters are used for initializing the members of gamma. *Arglist1* is the argument list for the constructor of a, and *arglist2* is the argument list for the constructor of b. *arglist1* and *arglist2* may or may not use the arguments from *arglist*. Remember, *a(arglist1)* and *b(arglist2)* are function calls and therefore the argument do not contain the data types.

They are simply variables or constants. Example:

```
gamma(int x, int y, float z) : a(x) , b(x,z)
{
    Assignment section(for ordinary other members)
}
```

We can use as many member objects as are required in a class. For each member object we add a constructor call in the initializer list. The constructors of the member objects are called in the order in which they are declared in the nested class.

Question Bank-UNIT III**Multiple choice questions**

1. What is the correct syntax for function declaration
 - A. Function_name(argument-list);
 - B. Return_type function_name(argument-list);**
 - C. Class_name function_name(argument-list);
 - D. operator_name function-name(argument-list);
2. Which of the following permits function overloading on C++?
 - A. type
 - B. number of arguments
 - C. type & number of arguments**
 - D. different function name
3. What is Overloaded functions in C++ oops ?
 - A. Functions preceding with virtual keywords.
 - B. Functions inherited from base class to derived class.
 - C. Two or more functions having same name but different number of parameters or type.**
 - D. Two or more functions having different name but same type of parameters.
4. How many types of inheritance are provided as OOP feature?
 - A. 4
 - B. 3
 - C. 2
 - D. 5**
5. Which among the following best defines single level inheritance?
 - A. A base class inheriting a derived class
 - B. A derived class inheriting a base class**
 - C. A derived class inheriting a nested class
 - D. A class which gets inherited by 2 classes
6. Which access type data gets derived as private member in derived class:
 - A. Private**
 - B. Public
 - C. Protected
 - D. Protected and Private
7. What is meant by multiple inheritance?
 - A. Deriving a base class from derived class
 - B. Deriving a derived class from base class
 - A. Deriving a derived class from more than one base class**
 - B. None of the mentioned

8. Which symbol indicates derivation of child class from base class
- A. ::
 - B. :
 - C. ;
 - D. |
9. A derived class with only one base class is called..... inheritance.
- A. **Single**
 - B. Multiple
 - C. Multilevel
 - D. Hierarchical
10. What facilitates Reusability of the code achieved in OOPs
- A. Dynamic binding
 - B. Encapsulation
 - C. **Inheritance**
 - D. Message Passing
11. Which of the below is the error code?
- A. **int test(int a) { }**
double test(int b){ }
 - B. int exam(int a) { }
 - C. int test(int a) { }
 - D. int test(int a) { }
12. Which among the following is true , if a base class is inherited in protected access mode?
- A. **Public and Protected members of base class becomes protected members of derived class**
 - B. Only protected members become protected members of derived class
 - C. Private, Protected and Public all members of base, become private of derived class
 - D. Only private members of base, become private of derived class
13. Which is the appropriate way to declare members in the class which are not intended to be inherited
- A. Public members
 - B. Protected members
 - C. **Private members**
 - D. Private or Protected members.

D. It is not possible

```
D. class student{ }; class test{ };
   class topper:public topper{}
```

D. One base class can be derived into only 2 classes

```
/*define class body*/
};
```

D. Not possible

19. How can you make the private members inheritable? A. By making their visibility mode as public only
B. By making their visibility mode as protected only
C. By making their visibility mode as private in derived class
D. It can be done both by making the visibility mode public or protected
20. Which kind of inheritance supports a child class to inherit properties from more than one base class.
A. Hierarchical
B. Hybrid
C. Multilevel
D. Multiple
21. Which among the below statements is not correct about private inheritance
A. The private members of a base class cannot be inherited to the derived class.
B. The protected members of a base class become private in a derived class.
C. The protected members of a base class stay protected in a derived class.
D. The public members of a base class become the private members of the derived class.
22. Class X, class Y and class Z are derived from class BASE. This is
A. Multiple
B. Multilevel
C. Hierarchical
D. Single
23. Which is the appropriate statement among the below mentioned?
A. Base class inherits some or all of the properties of the derived class.
B. Derived class inherits some or all of the properties of the base class.
C. Virtual class inherits some or all of the properties of the abstract class.
D. Base class inherits some or all of the properties of the virtual class.
24. Which among the below inheritance type, has the constructors executed in the order of inheritance.
A. Multipath
B. Multiple
C. Multilevel
D. Hierarchical
25. What will be the order of execution of base class constructors in the following method of inheritance?
class P: public M, public N {...};
A. M(); N(); P();
B. N(); M(); P();
C. P(); M(); N();
D. M(); P(); N();

Long Answer question

1. Make use of any three types of inheritance and explain the concept of inheritance with examples.
2. Develop a program using the concept of single inheritance and explain the underlying concept.
3. Develop a program using the concept of multilevel inheritance and explain the underlying concept.
4. Describe the syntax and example of multiple inheritance. Discuss when do we use such inheritance?
5. Describe the significance of hierarchical inheritance concept? Give two examples.

6. Survey on function overloading and the different ways to overload a function, with an suitable example program.
7. Analyse how does the visibility mode control the access of members in the derived class?
8. Can you distinguish between public and protected visibility mode by giving suitable examples for each.
9. Discuss about the ambiguity that arise in multiple inheritance? How can one resolve it?
10. Discuss about containership and how it differs from inheritance?
11. Analyse and explain about constructors in derived class
12. Justify why Making use of the concept virtual base classes is useful in inheritance?

UNIT-IV

POINTERS, VIRTUAL FUNCTIONS & POLYMORPHISM

CHAPTER 7: POINTERS

7.1 INTRODUCTION

A pointer is a variable whose value is the address of another variable. Like any variable or constant, you must declare a pointer before you can work with it. We know that a pointer is a derived data type that refers to another data variable by storing the variable's memory address rather than data.

7.2 DECLARING AND INITIALIZING POINTERS

We can declare a pointer variable similar to other variables in C++. Like C, the declaration is based on the data type of the variable it points to. The declaration of a pointer variable takes the following form:

data-type *pointer-variable;

Here, *pointer-variable* is the name of the pointer, and the *data_type* refers to one of the valid C++ data types, such as int., char, float, and so on. The *data_type* is followed by an asterisk(*) symbol known as **indirection operator or dereferencing operator**, which distinguishes a pointer variable from other variables to the compiler.

We can locate asterisk (*) immediately before the pointer variable, or between the data type and the pointer variable, or immediately after the data type. It does not cause any effect in the execution process.

The asterisk you used to declare a pointer is the same asterisk that you use for multiplication. However, in this statement the asterisk is being used to designate a variable as a pointer.

Following are the valid pointer declaration –

```
int *ip; // pointer to an integer
double *dp; // pointer to a double
float *fp; // pointer to a float
char *ch // pointer to character
```

The actual data type of the value of all pointers, whether integer, float, character, or otherwise, is the same, a long hexadecimal number that represents a memory address. The only difference between pointers of different data types is the data type of the variable or constant that the pointer points to. In C++, you cannot assign the address of a variable of one type to a pointer of another type. Consider this example:

```
int *ptr;
double d = 9;
ptr = &d; // Error: can't assign double* to int*
```

Example of using pointers:

```
#include <iostream.h>
#include <conio.h>
void main()
{
    int a, *ptr1, *ptr2;
    clrscr();
    ptr1 = &a;
    ptr2 = &ptr1 ;
    cout << "The address of a : " << ptr1 << "\n";
    cout << "The address of ptr1 : " << ptr2;
    cout << "After incrementing the address values:\n\n";
```

```
ptr1+=2;
cout << "The address of a: "<< ptr1<< "\n";
ptr2+=2;
cout << "The address of ptr1:" << ptr2 << "\n";
}
```

The memory location is always addressed by the operating system. The output of the program may vary depends on the system.

The address of a: 0x8fb6fff4

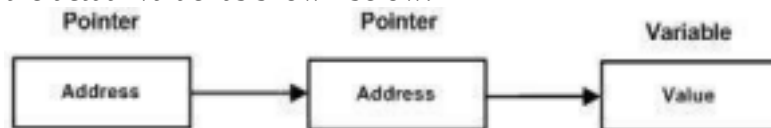
The address of ptr1:0x8fb6fff2

After incrementing the address values:

The address of a: 0x8fb6fff8

The address of ptr1: 0x8fb6fff6

A pointer to a pointer is a form of multiple indirection or a chain of pointers. Normally, a pointer contains the address of a variable. When we define a pointer to a pointer, the first pointer contains the address of the second pointer, which points to the location that contains the actual value as shown below.



A variable that is a pointer to a pointer must be declared as such. This is done by placing an additional asterisk in front of its name. For example, following is the declaration to declare a pointer to a pointer of type int –

```
int **var;
```

When a target value is indirectly pointed to by a pointer to a pointer, accessing that value requires that the asterisk operator be applied twice, as is shown below in the example –

```
#include <iostream.h>
```

```
int main ()
```

```
{
```

```
int var;
```

```
int *ptr;
```

```
int **pptr;
```

```
var = 3000;
```

```
// take the address of var
```

```
ptr = &var;
```

```
// take the address of ptr using address of operator &
```

```
pptr = &ptr;
```

```
// take the value using pptr
```

```
cout << "Value of var :" << var << endl;
```

```
cout << "Value available at *ptr :" << *ptr << endl;
```

```
cout << "Value available at **pptr :" << **pptr << endl;
```

```
return 0;
```

```
}
```

Output:

When the above code is compiled and executed, it produces the following result –

Value of var :3000

Value available at *ptr :3000

Value available at **pptr :3000

7.3 VOID POINTER

In C++, there is a general purpose pointer that can point to any type. This general purpose pointer is pointer to void. Void pointers are known as generic pointers, which refer to variables of any data type.

```
void *ptr; // pointer to void
```

```
int a = 10;
char b = 'x';
void *p = &a; // void pointer holds address of int 'a'
p = &b; // void pointer holds address of char 'b'
Void pointers cannot be dereferenced. For example the following program doesn't compile.
int main()
{
```

```
    int a = 10;
    void *ptr = &a;
    printf("%d", *ptr);
    return 0;
}
```

Error

Correct method: Before using void pointers, we must type cast the variables to the specific data types that they point to.

```
int main()
{
    int a = 10;
    void *ptr = &a;
    printf("%d", *(int *)ptr);
    return 0;
}
```

7.4 THE NULL POINTER VALUE

The pointers, which are not initialized in a program are called Null pointers. Pointers of any data type can be assigned with one value i.e '0' called null address.

This value can be expressed in C++ in two ways: either with an integer value of zero, or with the nullptr keyword:

```
int * p =0;
int * q = nullptr;
```

Here, both p and q are null pointers, meaning that they explicitly point to nowhere, and they both actually compare equal: all null pointers compare equal to other null pointers. It is also quite usual to see the defined constant NULL be used in older code to refer to the null pointer value:

```
int * r = NULL;
```

7.5 THIS POINTER

Every object in C++ has access to its own address through an important pointer called this pointer. The 'this' pointer is an implicit parameter to all member functions. Therefore, inside a member function, this may be used to refer to the invoking object.

Friend functions do not have a 'this' pointer, because friends are not members of a class. Only member functions have a 'this' pointer.

The 'this' pointer holds the address of current object, in simple words you can say that this pointer points to the current object of the class.

One of the important applications of using this pointer is to return the object it points. For example, the statement,

```
return *this;
```

inside a member function will return the object that calls the function.

Let us try the following example to understand the concept of this pointer –

```
#include <iostream.h>
class Box
{
public:
// Constructor definition
Box(double l = 2.0, double b = 2.0, double h = 2.0)
{
cout << "Constructor called." << endl;
length = l;
breadth = b;
height = h;
}
double Volume()
{
return length * breadth * height;
}
int compare(Box box)
{
return this->Volume() > box.Volume();
}
private:
double length; // Length of a box
double breadth; // Breadth of a box
double height; // Height of a box
};
int main()
{
Box Box1(3.3, 1.2, 1.5); // Declare box1
Box Box2(8.5, 6.0, 2.0); // Declare box2
if(Box1.compare(Box2)) {
cout << "Box2 is smaller than Box1" << endl;
}
else
{
cout << "Box2 is equal to or larger than Box1" << endl;
}
return 0;
}
```

Output:

Box2 is equal to or larger than Box1

7.6 POINTERS TO OBJECTS

A pointer can point to an object created by a class. Consider the following statement:

item x;

where item is a class and x is an object defined to be of type item.

Similarly we can define a pointer ptr of type item as follows:

item * ptr;

Object pointers are useful in creating objects at run time. We can also use an object pointer to access the public members of an object.

Consider a class item defined as follows:

```
#include <iostream.h>
class item
```



```
{
int code;
float price;
public:
void getdata(int a, float b)
{
code = a;
price = b;
void show(void)
{
cout << "code : " << code << "\n";
cout<< "Price: " << price << "\n";
}
};
const int size=2;
void main()
{
item *p =new item [size];
item *d=p;
int x, i;
float y;
for(i=0; i<size; i++)
{
cout << "Input code and price for item" << i+1;
cin>>x>>y;
p->getdata(x,y);
p++
for(i=0; i<size; i++)
{
cout<<"Item:"<<i+1;
d->show();
d++;
}
}
```

The output of Program will be:

Input code and price for item1: 40 500

Input code and price for item2: 50 600

Item: 1

Code : 40

Price: 500

Item:2

Code : 50

Price: 600

The statements

x.getdata(100,75.50);

x. show();

are equivalent to

p->getdata(100, 75.50);

p->show() ;

Since *p is an alias of x, we can also use the following method:

(*p). show();

The parentheses are necessary because the dot operator has higher precedence than the *indirection operator* *.

We can also create the objects using pointers and new operator as follows:

```
item *p = new item;
```

This statement allocates enough memory for the data members in the object structure and assigns the address of the memory space to p. Then p can be used to refer to the members as shown below: `p -> show();`

We can also create an array of objects using pointers. For example, the statement

```
item *p = new item[10]; // array of 10 objects
```

creates memory space for an array of 10 objects of item. Remember, in such cases if the class contains constructors, it must also contain an empty constructor.

7.7 ARRAY OF OBJECTS USING POINTERS

- We can also create an array of objects using pointers. For example, the statement `item *ptr = new item[10]; // array of 10 objects`
- Creates memory space for an array of 10 objects of item.
- Remember, in such cases if the class contains constructors, it must also contain an empty constructor.

```
class item
{
int code;
float price;
public:
void getdata(int a, float b)
{
code=a;
price = b;
}
void show()
{
cout << "code = " << code << "\n";
cout<< "Price: " << price << "\n";
}
};
const int size=2;
void main()
{
item *p=new item [size];
item *d= p;
int x, i;
float y;
for(i=0; i<size; i++)
{
cout << "input code and price for item" << i+1;
cin >> x >> y;
p->getdata(x,y);
p++;
}
for(i=0; i<size; i++)
{
cout<<"item="<<i+1;
d ->show();
d++;
}
```

```
}  
return 0;  
}
```

Output:

Input code and price for item1: 40 500

Input code and price for item2 :50 600

Item: 1

Code : 40

Price: 500

Item:2

Code : 50

Price: 600

7.8 POINTERS TO DERIVED CLASSES

We can use pointers not only to the base object but also to the objects of derived classes. Pointers to objects of a base class are type-compatible with pointers to objects of derived class

Therefore, a single class pointer variable can be made available to point to objects belonging to different classes.

For example, if B is a base class and D is a derived class from B, then a pointer declared as a pointer to B can also be a pointer to D.

```
B *cptr;
```

```
B b;
```

```
D d;
```

```
cptr =&b;
```

We can make cptr to point the object d as follows:

```
cptr=&d; // this is valid because d is an object derived from B.
```

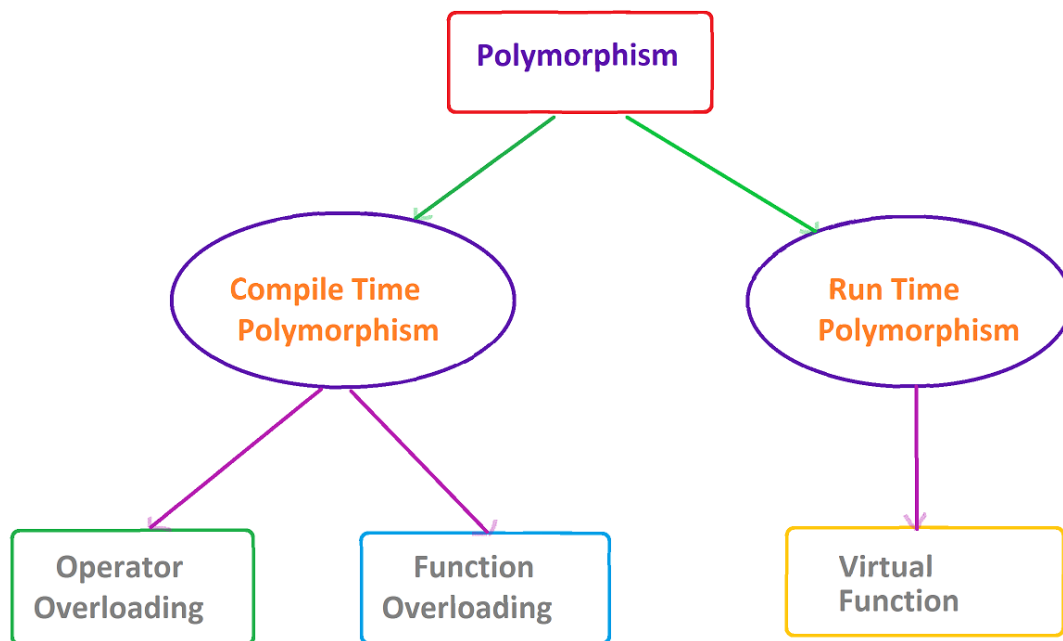
- However there is a problem in using cptr to access the public member of the derived class D .
- Using cptr, we can access only those members which are inherited from B and not the members that originally belong to D .
- In case a member of D has the same name as one of the members of B, then any reference to that member by cptr will always access the base class member.
- Although C++ permits a base pointer to point to any object derived from that base, the pointer cannot be directly used to access all the members of the derived class.
- We may have to use another pointer *declared as pointer to the derived type*.

CHAPTER 8 POLYMORPHISM

8.1 INTRODUCTION

Polymorphism is one of the crucial features of OOP. Polymorphism is implemented using overloaded functions and operators. The overloaded member functions are selected for invoking by matching arguments, both type and number.

- It simply means 'one name, multiple forms'.
- It is classified into 2 categories.



8.1.1 COMPILE-TIME POLYMORPHISM

- Means that an object is bound to its function call at the compile time.
- No ambiguity at the compile time about which function is to be linked to a particular function's call.
- Also known as **early binding** or **static binding** or **static linking**.

It is achieved in 2 ways:

- Function overloading and operator overloading.

8.1.2 RUN TIME POLYMORPHISM

- The uses of scope resolution operator in resolving ambiguity works fine in smaller programs.

b;

b.display();

b.A::display();

- But in large applications, it would be tedious to modify the code which implements static binding.
- To overcome this problem, run-time polymorphism is very useful.
- It links the function call to a particular class at run-time.
- Thus, it is not known which function will be invoked till an object actually makes the function call during the program's execution.
- **This process is known as late binding or dynamic binding.**
- Run-time polymorphism is achieved through **virtual functions**.

8.2 FUNCTION OVERRIDING IN C++

- **Function overriding** is a feature that allows us to have a same function in child class which is already present in the parent class.

- A child class inherits the data members and member functions of parent class, but when we want to override a functionality in the child class then we can use function overriding.
- It is like creating a new version of an old function, in the child class.
- Suppose, both base class and derived class have a member function with same name and arguments (number and type of arguments).
- If we create an object of the derived class and call the member function which exists in both classes (base and derived), the member function of the derived class is invoked and the function of the base class is ignored.

```
#include <iostream.h>
```

```
class BaseClass
```

```
{
```

```
public: void disp()
```

```
{
```

```
cout<<"Function of Parent Class";
```

```
}
```

```
};
```

```
class DerivedClass: public BaseClass
```

```
{
```

```
public: void disp()
```

```
{
```

```
cout<<"Function of Child Class";
```

```
}
```

```
};
```

```
void main() {
```

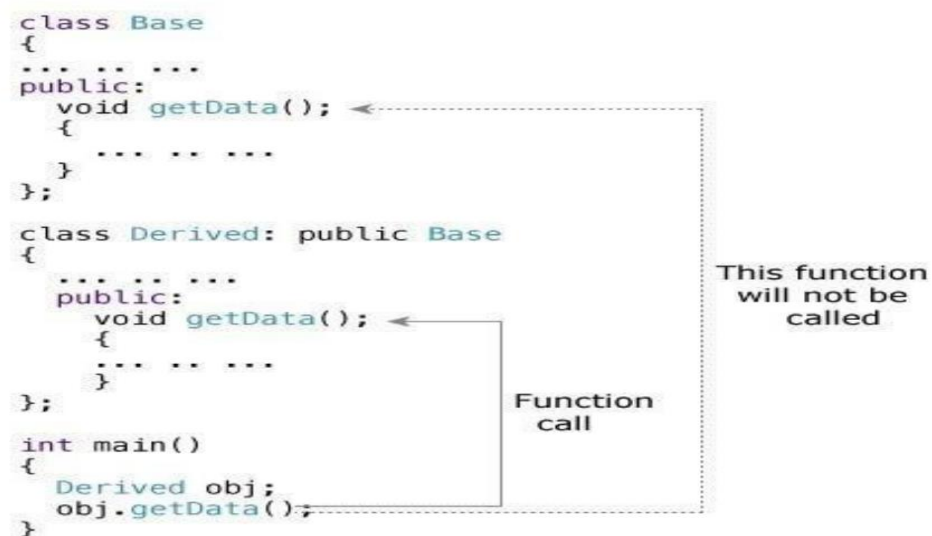
```
DerivedClass obj;
```

```
obj.disp();
```

```
}
```

Output: Function of Child Class

Note: In function overriding, the function in parent class is called the overridden function and function in child class is called overriding function.



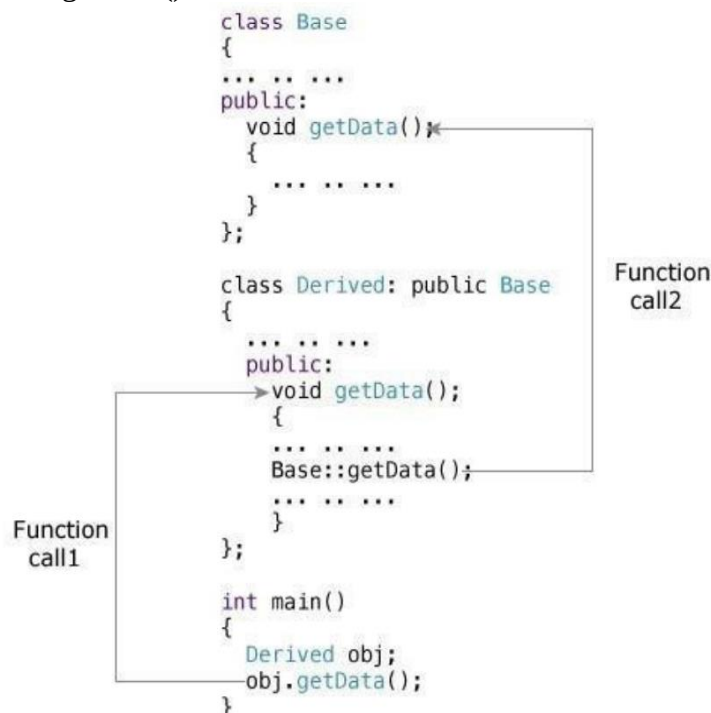
8.2.1 How to access the overridden function in the base class from the derived class?

• To access the overridden function of the base class from the derived class, scope resolution operator :: is used. For example,

- If you want to access getData() function of the base class, you can

use the following statement in the derived class.

- `Base::getData();`



8.2.2 Requirements for Overriding a Function

- Inheritance should be there.
- Function overriding cannot be done within a class. For this we require a derived class and a base class.
- Function that is redefined must have exactly the same declaration in both base and derived class, that means same name, same return type and same parameter list.

8.3 VIRTUAL FUNCTIONS

- A virtual function is a member function which is declared within **base** class and is re-defined (Overridden) by derived class.
- When we refer to a derived class object using a pointer or a reference to the **base** class, we can call a virtual function for that object and execute the derived class's version of the function.
- Virtual functions ensure that the correct function is called for an object, regardless of the type of reference (or pointer) used for function call.
- They are mainly used to achieve **run-time polymorphism**.
- Functions are declared with a **virtual** keyword in base class.
- The resolving of function call is done at Run-time.

The keyword `virtual` is used for defining virtual function.

```

class class_name
{
public:
    virtual return func_name( args.. )
    {
        //function definition
    }
}

```

8.3.1 PROBLEM WITHOUT VIRTUAL KEYWORD

Let's try to understand what is the issue that virtual keyword fixes,
class Base

```
{
public: void show() { cout << "Base class"; }
};
class Derived:public Base
{
public: void show() { cout << "Derived Class"; }
};
void main()
{
Base b; //Base class object
Base *bptr; //base class pointer
Derived d; //Derived class object
bptr=&b;
bptr-> show(); //calls base class
bptr = &d;
bptr->show(); //calls base class, early binding
}
```

When we use Base class's pointer to hold Derived class's object, base class pointer or reference will always call the base version of the function.

8.3.2 USING VIRTUAL KEYWORD IN C++

We can make base class's methods virtual by using **virtual** keyword while declaring them. Virtual keyword will lead to Late Binding of that method.

```
class Base
{
public: virtual void show()
{
cout << "Base class\n";
}
};
class Derived:public Base
{
public: void show()
{
cout << "Derived Class";
}
};
void main()
{
Base* b;
Derived d;
b = &d;
b->show();
}
```

- On using Virtual keyword with Base class's function, Late Binding takes place and the derived version of function will be called, because base class pointer points to Derived class object. **Example 2:**

```
#include<iostream.h>
#include<conio.h>
class base
{
public:
virtual void show()
```

```
{
cout << "\n Base class show:";
}
void display()
{
cout << "\n Base class display:";
}
};
class drive : public base
{
public: void display()
{
cout << "\n Drive class display:";
}
void show()
{
cout << "\n Drive class show:";
}
};
void main()
{
clrscr();
base obj1;
base *p;
cout << "\n P points to base:\n";
p = &obj1;
p->display();
p->show();
cout << "\n P points to drive:\n";
drive obj2;
p = &obj2;
p->display();
p->show();
getch();
}
```

Output:

```
P points to Base
Base class display
Base class show
P points to Drive
Base class Display
Drive class Show
```

Using Virtual Keyword and Accessing Private Method of Derived class

- We can call **private** function of derived class from the base class pointer with the help of **virtual** keyword. Compiler checks for access specifier only at compile time. So at run time when late binding occurs it does not check whether we are calling the private function or public function.

8.4 RULES FOR VIRTUAL FUNCTIONS

1. The virtual functions must be members of some class.
2. They cannot be static members.
3. They are accessed by using object pointers.
4. A virtual function can be a friend of another class.

5. A virtual function in a base class must be defined, even though it may not be used.
6. The prototypes of the base class version of a virtual function and all the derived class versions must be identical. If two functions with the same name have different prototypes, C++ considers them as overloaded functions, and the virtual function mechanism is ignored.
7. We cannot have virtual constructors, but we can have virtual destructors.
8. While a base pointer can point to any type of the derived object, the reverse is not true. That is to say, we cannot use a pointer to a derived class to access an object of the base type.
9. When a base pointer points *to a derived class, incrementing or decrementing it will not make it to point to the next object of the derived class*. It is incremented or decremented only relative to its base type. Therefore, we should not use this method to move the pointer to the next object.
10. If a virtual function is defined in the base class, it need not be necessarily redefined in the derived class. In such cases, calls will invoke the base function.

8.5 PURE VIRTUAL FUNCTIONS

- It is normal practice to declare a function virtual inside the base class and redefine it in the derived classes.
- The function inside the base class is not used for performing any task. It only serves as a placeholder.
- For example,

```
class media
{
.....
.....
virtual void display() //empty virtual function
{
}
};
```

- If we have not defined any object of class **media** and therefore the function display() in the base class has been defined 'empty'.
 - Such functions are called "do-nothing" functions.
- A "do-nothing" function may be defined as follows: virtual void display() =0;
- Such functions are called *pure virtual functions*.
- A pure virtual function is a function declared in a base class that has no definition relative to the base class.
- In such cases, the compiler requires each derived class to either define the function or redeclare it as a pure virtual function.
- Remember that a class containing pure virtual functions cannot be used to declare any objects or its own.
- Such class are called **abstract base classes**.
- The main objective of an abstract base class is to provide some properties to the derived classes and to create a base Pointer required for achieving run time polymorphism.

```
class A
{
public: virtual void example()=0;
};
class B: public A
{
```

```
public: void example()
{
cout<<"Text book C++ written by A";
}
};
class C: public A
{
public: void example()
{
cout<<"Text book Java written by A";
}
};
void main()
{
A *ptr[2]; B b;
C c; ptr[0]=&b;
ptr[1]=&c;
ptr[0]->example();
ptr[1]->example();
}
```

QUESTION BANK IV**Multiple Choice questions**

1. which symbol is used to declare the pointer variable.
A. @ **B. *** C. # D. ()
2. Polymorphism means
A. **Many forms**
B. Only one form
C. Hiding data
D. None of them
3. Compile time polymorphism is also known as.....
A. late binding
B. **Static binding**
C. Dynamic binding
D. Fixed binding
4. The pointers which are not initialized in a program are called A. void pointers
B. **null pointers**
C. this pointer
D. base pointer
5. which pointers are known as generic pointers.
A. This pointer
B. Null pointer
C. **Void pointer**
D. Base pointer
6. Which pointer is useful in creating objects at run time.
A. void pointer
B. null pointer
C. this pointer
D. **object pointer**
7. A pure virtual function is initialized by
A. **0**
B. 1
C. 2
D. 3
8. Which of the following keyword is used before a function in a base class to be overridden in derived class in C++
A. override
B. **virtual**
C. void
D. none

9. Which type of function among the following shows polymorphism? A. Inline function
B. Undefined functions
C. Virtual function
D. Class member functions
10. A pointer can be initialized with
A. Null
B. Zero
C. Address of an object of same type
D. All of the above
11. Examine the below instructions and identify the correct initialization statement in pointers?
A. ptr=&a; B. ptr=a; C. ptr=*a D. ptr=@a;
12. Examine the below and identify the correct declaration of an array of pointers?
A. int &array[10];
B. int *array[10];
C. int a[*10];
D. int array[10];
13. can you analyze the following statements about virtual functions and state Whether True or False.
i) A virtual function, equated to zero is called pure virtual function. ii) A class containing pure virtual function is called an abstract class **A. True, True**
B. True, False
C. False, True
D. False, False
14. Can you judge which is the correct declaration of pure virtual function in C++
A. virtual void func = 0;
B. virtual void func() = 0;
C. virtual void func(){0};
D. void func() = 0;
15. Can you identify which pointer denotes the object calling the member function?
A. Variable pointer
B. this pointer
C. Null pointer
D. Zero pointer
16. Select the mechanism which means that an object is bound to its function call at the compile time.
A. Inheritance B. Message Passing
C. Compile-time polymorphism D. Function Overriding
17. Judge on which of the following statements are true according virtual functions?
i) The virtual functions must be members of some class.
ii) A virtual function can be a friend of another class
A. Only i **B. Both i and ii** C. Both are wrong D. Only ii

18. Select the polymorphism mechanism involving Function overriding **A. run-time** B. compile-time C. Static D. Object

19. Select the function declared in a base class that has no definition relative to the base class.

- A. member function
- B. virtual function
- C. pure virtual function**
- D. pure function

20. Suppose a polymorphism is supported in C++ with the help of virtual function, then it is termed as?

- A. Dynamic binding**
- B. run time
- C. early binding
- D. static

21. Suppose a polymorphism has the object is bound to its function call at compile time, then it can be termed as?

- A. Dynamic binding**
- B. run time
- C. early binding**
- D. easy binding

22. Can you assess which among the following best describes polymorphism? **A. It is the ability for a message/data to be processed in more than one form** B. It is the ability for a message/data to be processed in only 1 form

- C. It is the ability for many messages/data to be processed in one way
- D. It is the ability for undefined message/data to be processed in at least one way

23. Assess which among these is true about abstract base class?

- A. It can have One or more pure virtual member functions**
- B. It can have One or more child classes
- C. It can have One or more normal functions
- D. It can have one or more objects

24. Suppose derived class defines same function as defined in its base class, then such a situation is termed as?

- A. Function overloading B. Member function
- C. Inheritance **D. Function Overriding**

25. Examine the given code and Choose the right option from below? `string *x, y;`

- A. x is a pointer to a string, y is a string**
- B. y is a pointer to a string, x is a string
- C. both x and y are pointer to string types
- D. both x and y are string to pointer types

Long Answer questions

1. Make use of the concept of pointers and show how to declare and initialize a pointer and also define a pointer?
2. Develop a program to explain the concept of pointers to object
3. Analyze on what does 'this' pointer points to?
4. Elaborate on how polymorphism can be achieved at i) compile time ii) run time?
5. Elaborate on function overriding in c++ with an example.
6. Analyze the define the meaning and need of virtual function?
7. Can you identify and determine any four rules for virtual functions.
8. What is the motive of making a virtual function 'pure'? Explain.
9. Analyze and explain Array of Objects using Pointers.
10. Plan on how to access the overridden function in the base class from the derived class? Explain.
11. Discuss the following by giving examples.
 - i) void pointers
 - ii) null pointer value
12. Analyze on how to use the pointers to objects of the derived class?

UNIT V**TEMPLATES & EXCEPTION HANDLING CHAPTER 9:
TEMPLATES****INTRODUCTION**

Templates are powerful features of C++ which allows us to write generic classes and functions thus provide support for generic programming.

Generic programming is an approach where generic types are used as parameters in algorithms so that they work for a variety of suitable data types and data structures.

In simple terms, we can create a single function or a class to work with different data types using templates.

A template can be used to create a family of classes or functions. For example, a class template for an array class would enable us to create arrays of various data types such as int array and float array.

- Similarly we can also define a template for a function, say mul(), that would help us create various versions of mul() for multiplying int, float and double type values.

Templates are often used in larger codebase for the purpose of code reusability and flexibility of the programs.

Templates are of great utility to programmers in C++, especially when combined with multiple inheritance and operator overloading.

The C++ Standard Library provides many useful functions within a framework of connected templates.

The concept of templates can be used in two different ways:

- Class Templates
- Function Templates

CLASS TEMPLATES

- We can create class templates for generic class operations.

Sometimes, we need a class implementation that is same for all classes, only the data types used are different.

Normally, we would need to create a different class for each data type OR create different member variables and functions within a single class.

- Due to this we need to write a large application and will be hard to maintain, as a change in one class/function should be performed on all classes/functions.
- Class templates make it easy to reuse the same code for all data types.

HOW TO DECLARE A CLASS TEMPLATE?

Syntax:

```
template <class  
T>      class  
className  
{  
... ..  
public: T a;  
T someOperation(T arg);  
... ..  
};
```

The class template definition is similar to an ordinary class definition except the prefix template

<class T> and the use of type T.

In the above declaration, T is the template argument which is a placeholder for the data type used.

- Inside the class body, a member variable a and a member function someOperation() are both of type T.

A class created from a class template is called template class. The syntax for defining an object of a template class is:

classname <data type> objectname(arglist);

This process of creating a specific class from a class template is called instantiation. For example,

```
className<int> classObject;
```

```
className<float>
```

```
classObject;
```

```
className<string>
```

```
classObject;
```

Example:

Program to add, subtract, multiply and divide two numbers using class template. #include <iostream.h>

```
template <class T>class Calculator
```

```
{
```

```
private: T num1, num2; public: Calculator(T n1, T n2)
```

```
{
```

```
num1 = n1; num2 = n2;
```

```
}
```

```
void displayResult()
```

```
{
```

```
cout << "Numbers are: " << num1 << " and " << num2 << endl;
```

```
cout << "Addition is: " << add() << endl;
```

```
cout << "Subtraction is: " << subtract() << endl;
```

```
cout << "Product is: " << multiply() << endl;
```

```
cout << "Division is: " << divide() << endl;
```

```
}
```

```
T add() { return num1 + num2; }
```

```
T subtract() { return num1 - num2;
```

```
} T multiply() { return num1 *
```

```
num2; } T divide() { return num1 /
```

```
num2; }
```

```
};
```

```
void main()
```

```
{
```

```
Calculator<int> intCalc(6, 2);
```

```
Calculator<float> floatCalc(2.4,
```

```
3.2); cout << "Int results:" << endl;
```

```
intCalc.displayResult();
```

```
cout << endl << "Float results:" << endl;
```

```
floatCalc.displayResult();
```

```
}
```

Output:

Int results:

Numbers are: 6 and 2

Addition is: 8
Subtraction is 4
Product is: 12
Division
is: 3 Float
Results
Numbers are 2.4 and 3.2
Addition is: 5.6
Subtraction is :-0.8
Product is: 7.68
Division is: 0.75

CLASS TEMPLATE WITH MULTIPLE PARAMETERS

We can use more than one generic data types in a class template.

They are declared as,

```
template <class T1,class T2, ..... >class className
{
... ..
.....(Body of the class)
.....
};
```

Example: #include<iostream.h>

```
Template<class T1, class T2>
```

```
class Test
```

```
{
```

```
T1 a;T2 b;
```

```
public: Test(T1 x, T2 y)
```

```
{
```

```
}
```

```
void show()
```

```
{
```

```
a=x;b=y;
```

```
cout<<a<<" and" <<b<<"\n";4
```

```
}
```

```
};
```

```
void main()
```

```
{
```

```
Test<float, int> test1(1.23, 8);
```

```
Test<int,char> test2(100, 'w');
```

```
test1.show();
```

```
test2.show();
```

```
}
```

Output:

1.23 and 8

100 and w

FUNCTION TEMPLATES

- A function template works similar to a normal function with one key difference.
- A single function template can work with different data types at once but, a single normal function can only work with one set of data types.

- Normally, if you need to perform identical operations on two or more types of data, you use function overloading to create two functions with the required function declaration.
- However, a better approach would be to use function templates because you can perform the same task writing less and maintainable code.

DECLARING A FUNCTION TEMPLATE

- A function template starts with the keyword `template` followed by template parameter/s inside `< >` which is followed by function declaration.
- In the above code, `T` is a template argument that accepts different data types (`int`, `float`), and `class` is a keyword.
- You can also use keyword `typename` instead of `class` in the above example.
- When, an argument of a data type is passed to `someFunction( )`, compiler generates a new version of `someFunction()` for the given data type.

Example 1: Function Template to find the largest number

- Program to display largest among two numbers using function templates.

// If two characters are passed to function template, character with larger ASCII value is displayed.

```
#include <iostream.h>
template <class T>
T Large(T n1, T n2)
{
    return (n1 > n2) ? n1 : n2;
}

void main()
{
    int i1, i2;
    float f1, f2;
    char c1, c2;
    cout << "Enter two integers:\n"; cin >> i1 >> i2;
    cout << Large(i1, i2) << " is larger." << endl;
    cout << "\nEnter two floating-point numbers:\n"; cin >> f1 >> f2;
    cout << Large(f1, f2) << " is larger." << endl;
    cout << "\nEnter two characters:\n";
    cin >> c1 >> c2;
    cout << Large(c1, c2) << " has larger ASCII value.";
}
```

Output

```
Enter two integers: 5 10
10 is larger.
```

Enter two floating-point numbers: 12.4 10.2
12.4 is larger.
Enter two characters:
z Z z has larger ASCII
value.

Example 2:

Program to swap data using function templates. #include <iostream.h>

```
template <typename T>
```

```
void Swap(T &n1, T &n2)
```

```
{
```

```
T temp;
```

```
temp = n1;
```

```
n1 = n2;
```

```
n2 = temp;
```

```
}
```

```
void main()
```

```
{
```

```
int i1 = 1, i2 = 2;
```

```
float f1 = 1.1, f2 = 2.2; char c1 = 'a', c2 = 'b';
```

```
cout << "Before passing data to function template.\n";
```

```
cout << "i1 = " << i1 << "\ni2 = " << i2;
```

```
cout << "\nf1 = " << f1 << "\nf2 = " << f2;
```

```
cout << "\nc1 = " << c1 << "\nc2 = " << c2;
```

```
Swap(i1, i2);
```

```
Swap(f1, f2);
```

```
Swap(c1, c2);
```

```
cout << "\n After passing data to function template.\n";
```

```
cout << "i1 = " << i1 << "\ni2 = " << i2;
```

```
cout << "\nf1 = " << f1 << "\nf2 = " << f2; cout << "\nc1 = " << c1 << "\nc2 = " << c2;
```

```
}
```

Output:

Before passing data to function template i1=1 i2=2

f1=1.2 f2=2.2 c1 = 'a', c2 = 'b';

After passing data to function template i1=2 i2=1

f1=2.2 f2=1.2 c1 = 'b', c2 = 'a';

FUNCTION TEMPLATE WITH MULTIPLE PARAMETERS

Like template classes, we can use more than one generic data type in the template statement,

using a comma-separated list.

```
template <class T1, class T2,..... >
```

```
returntype functionname(argmts of T1, T2 .... )
```

```
{
```

```
.....
```

```
..... (body of function)
```

```
.....
```

```
}
```

Example program

```
#include<iostream.h>
#include<string.h>
Template<class T1, class T2>
void display(T1 x, T2 y)
{
cout<<x << "and"<<y<<"\n";
void main()
{
cout<<" Function template with int and string type"<<"\n";
display(1999, "ABC");
cout<<"Function template with float and integer type";
display(12.34, 1234);
}
```

OVERLOADING OF TEMPLATE FUNCTIONS

- A template function can be overloaded either by template functions or ordinary functions of its name.

An overloading is accomplished as follows:

1. Call an ordinary function that has an exact match.
2. Call a template function that could be created with an exact match.
3. Try normal overloading to ordinary functions and call the one that matches. An error is generated if no match is found.

Example program:

```
#include<iostream.
h>
#include<string.h>
template<class T1>
void display(T1 x)
{
cout<< "Overloaded Template display 1:" <<x<<"\n";
}
template <class T1 x, T2 y)
void display(T1 x, T2 y)
{
cout<<"Overloaded template display 2:"<<x<<"and" <<y;
}
void display(int x)
{
cout<< "Explicit display" <<x<<"\n";
}
void main()
{
display(100); display(10, 20);
```

```
display(12.34); display(2.45, 5.5);  
display('C'); display('A', 'B');  
}
```

MEMBER FUNCTION TEMPLATES

- This means defining the member functions of template class outside the class.
- Member functions of template classes are parameterized by the type argument (to their template classes) and these functions must be defined by the function templates. General form:

```
Template <class T>  
returntype classname<T> :: functionname (arglist)  
{  
..... // function body  
}
```

CHAPTER -10 EXCEPTION HANDLING

INTRODUCTION

- We know that it is very rare that a program works correctly first time.

It might have bugs. The two most common types of bugs are, Logic errors and syntactic errors.

- We can detect these errors by using debugging and testing procedures.

We often come across some problems other than logic or syntax errors. They are known as exceptions.

Exceptions are run time anomalies or unusual conditions that a program may encounter while executing.

- Anomalies might include conditions such as division by zero, access to an array outside of its bounds, or running out of memory or disk space.

BASICS OF EXCEPTIONAL HANDLING

There are two kinds of exceptions:

- **SYNCHRONOUS EXCEPTIONS:** The exceptions which occur during the program execution due to some fault in the input data or technique that is not suitable to handle current

class of data, within the program are known as synchronous exceptions. Errors such as out-of-range index, “over-flow” and underflow belong to this.

Exception

handling is designed to support only synchronous exceptions, such as array range checks. The

term synchronous exception means that exceptions can be originated only from throw expressions.

The C++ standard supports synchronous exception handling with a termination model. Termination means that once an exception is thrown, control never returns to the throw point.

- **ASYNCHRONOUS EXCEPTIONS:** The exceptions are caused by events or faults unrelated (External) to the program and beyond the control of the program are called asynchronous exceptions. For example, errors such as keyboard interrupts, hardware malfunctions, disk failure etc.

Exception handling is not designed to directly handle asynchronous exceptions such as keyboard interrupts. However, we can make exception handling work in the presence of asynchronous events if we are careful.

For instance, to make exception handling work with signals, we can write a signal handler that sets a global variable, and create another routine that polls the value of that variable at regular intervals and throws an exception when the value changes. We cannot throw an exception from a signal handler.

The purpose of exception handling mechanism is to provide means to detect and report an exception so that proper action can be taken.

Error handling mechanism performs following tasks:

1. Find the problem (Hit the exception)
2. Inform that an error has occurred (Throw the exception)
3. Receive the error information (Catch the exception)

4. Take corrective actions (handle the exception)

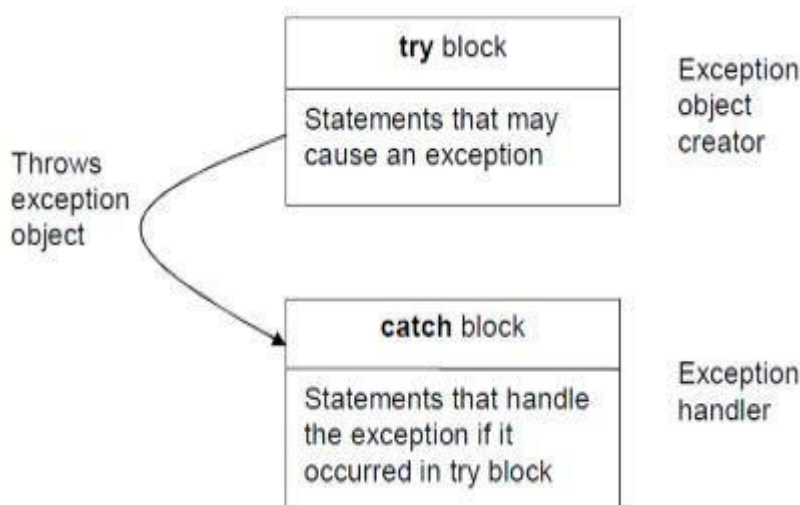
Error handling mechanism consists of two segments, one to detect errors and to throw exceptions, and the other to catch the exceptions and to take appropriate actions.

EXCEPTION HANDLING MECHANISM

C++ exception handling mechanism built upon three keywords: try, throw and catch. The keyword try is used to preface a block of statements which may generate exceptions.

This block of statements is known as try block.

When an exception is detected, it is thrown using a throw statement in the try block. A catch block defined by the keyword catch 'catches' the exception 'thrown' by the throw statement in the try block and handles it properly.



The catch block that catches an exception must immediately follow try block that throws the exception. The general form of these two blocks is as follows:

```

.....
.....
try
{
    .....
    throw exception;           // Block of statements which
    .....                     // detects and throws an exception
    .....
}
catch(type arg)               // Catches exception
{
    .....
    .....                     // Block of statements that
    .....                     // handles the exception
    .....
}
.....
.....
  
```

- When the try block throws an exception, the program control leaves the try block and enters the catch statement of the catch block.
- Note that exceptions are objects used to transmit information about a problem.
- If the type of object thrown matches the arg type in the catch statement, then catch

block is
executed for handling the exception.

- If they do not match, the program is aborted with the help of abort() function which is involved by default.

When no exception is detected and thrown, the control goes to the statement immediately after the catch block. i.e the catch block is skipped.

```
#include<iostream.h>
void main()
{
    int a,b;
    cout<<"Enter the values of a and b\n";
    cin>>a>>b;
    int x=a-b;
    try
    {
        if(x!=0)
        {
            cout<<"Result (a/x)="<<a/x;
        }
        else
        throw(x);
    }
    catch(int i)
    {
        cout<<"Exception caught: Divide by Zero\n";
    }
    cout<<"End";
}
```

Output:

First Run

Enter the values of a and b: 20 15 Result(a/x)=4

Second Run

Enter the values of a and b: 10 10

Exception caught: Divide by Zero

- The output of the first run shows successful execution. When no exception is thrown, the catch block is skipped and execution resumes with the first line after catch.
- In second run, the denominator x becomes zero and therefore a division-by-zero situation occurs.
- This exception is thrown using the object x.
- Since the exception object is an int type, the catch statement containing int type argument catches the exception and displays necessary message.
- Exceptions are thrown by functions that are invoked from within the try blocks.
- The point at which throw is executed is called the throw point.
- Once an exception is thrown to the catch block, control cannot return to the throw point.


```

Type function(arg list)           // Function with exceptions
{
    .....

    .....
    throw( object);               // Throws exception
    .....
}
.....

.....
                                ...
                                ...
                                .
.....
t      ..... Invoke function here
.....      r
                                y
{

}
catch(type arg)
{

    ..... Handles exception here

}
.....

```

Note: The try block is immediately followed by the catch block, irrespective of the location of the throw point.

Below program demonstrates how a try block invokes a function that generates an exception.

Note: The try block is immediately followed by the catch block, irrespective of the location of the throw point.

Below program demonstrates how a try block invokes a function that generates an exception.

```

INVOKING FUNCTION THAT GENERATES EXCEPTION
// Throw point outside the try block
#include <iostream>
using namespace std;
void divide(int x, int y, int z)
{
    cout << "\nWe are inside the function \n";
    if((x-y) != 0)           // It is OK
    {
        int R = z/(x-y);
        cout << "Result = " << R << "\n";
    }
    else                    // There is a problem
    {
        throw(x-y);        // Throw point
    }
}

int main()
{
    try
    {
        cout << "We are inside the try block \n";
        divide(10,20,30);    // Invoke divide()
        divide(10,10,20);    // Invoke divide()
    }
    catch(int i)             // Catches the exception
    {
        cout << "Caught the exception \n";
    }
    return 0;
}

```

Output:

We are inside the try blockWe are inside the functionResult=-3
We are inside the functionCaught the exception

THROWING MECHANISM

When an exception that is desired to be handled is detected, it is thrown using the throw statement

in one of the following forms:

throw(exception);

throw exception;

throw; //used for re-throwing an exception

The operand object exception may be of any type, including constants. It is also possible to throw objects not intended for error handling.

When an exception is thrown, it will be caught by the catch statement associated with the try block.

That is, the control exists the current try block, and is transferred to the catch block after that try block.

Throw point can be in a deeply nested scope within a try block or in a deeply nested function call. In any case, control is transferred to the catch statement.

CATCHING MECHANISM

- Code for handling exceptions is included in catch blocks. A catch block looks like a function definition.

```
catch( type arg)
```

```
{
```

```
//statements for managing exceptions
```

```
}
```

The type indicates the type of exception that catch block handles. The parameter arg is an optional parameter name. When the exception is caught, the code in the catch block is executed.

MULTIPLE CATCH STATEMENTS

It is possible that a program segment has more than one condition to throw an exception. In such

cases, we can associate more than one catch statement with a try as shown below:try

```
{
//try block
}
catch(type1 arg)
{
// catch block1
}
catch(type2 arg)
{
// catch block2
}
.....
.....
catch(typeN arg)
{
// catch blockN
}
```

- When an exception is thrown, the exception handlers are searched in order for an appropriate match.

- The first handler that yields a match is executed.

After executing the handler, the control goes to the first statement after the last catch block for that try.

- When no match is found, the program is terminated.

It is possible that arguments of several catch statements match the type of an exception. In such cases, the first handler that matches the exception type is executed.

Exception Handling with Multiple Catch Example Program:

```
#include<iostream.h>
#include<conio.h>
void test(int x)
{
try
{
if (x > 0) throw x; //int
else
throw 'x'; //char
}
catch (int x)
{
cout << "Catch a integer and that integer is:" << x;
}
}
```

```
catch (char m)
{
cout << "Catch a character and that character is:" << m;
}
}
void main()
{
cout << "Testing multiple catches\n";test(10);
test(0);
getch();
}
```

Output:

Testing multiple catches Catch a integer and
that integer is: 10
Catch a character and that character is: x

CATCH ALL EXCEPTIONS

There is a special catch block called ‘catch all’

catch(...) that can be used to catch all types of exceptions. For
example, in the following program,

an int is thrown as an exception,

but there is no catch block for int, so catch(...) block will be executed.

```
#include<iostream.h>
```

```
#include<conio.h>
```

```
void test(int x)
```

```
{
try
{
if (x >0) throw x; //int
else if(x==0) throw ‘x’; //char
else throw 1.0; //float
}
}
```

```
Catch()
```

```
{
cout << “caught an exception\n” ;
}
```

```
void main()
```

```
{
cout << "Testing all exceptions\n";
test(1);
test(0);
test(-1);
getch();
}
```

Output:

Testing all exceptions
Caught an exception
Caught an exception
Caught an exception

Note:

- All throws were caught by the catch(...) statement.
- catch(...) should always be placed last in the list of handlers.

RETHROWING AN EXCEPTION

- An exception handler may decide to throw the exception caught without processing it.
- In such situations, we invoke throw without any arguments as shown below:
throw;
• This causes the current exception to be thrown to the next enclosing try/catch sequence and is caught by catch statement listed after that enclosing try block.

```
#include<iostream.h>
void divide(double x, double y)
{
    cout<<"Inside
function\n";try
{
    if(y==0.0)
        throw y; //throwing double
    else
        cout<<"Division = "<<x/y;
    }
    catch(double)
    {
        cout<<"Caught double inside function";
        throw; //rethrowing double
    }
    cout<<" end of function";
}
void main()
{
    cout<<"Inside
main";try
{
    divide(10.5,2.0);
    divide(20.0,0.0);
}
    catch(double)
    {
        cout<<"caught double inside main";
    }
    cout<<"End of main";
}
```

Output:

```
Inside main
Inside function
Division=5.25
End of function
Inside function
```

Caught double inside function
Caught double inside main
End of main

SPECIFYING EXCEPTIONS

It is possible to restrict a function to throw only certain specified exceptions. This is achieved by adding a throw list clause to the function definition.

The general form of using an exception specification is

```
type function(arglist) throw (type-list)
{
.....
..... //function body
}
```

The type-list specifies the type of exceptions that may be thrown.

Throwing any other type of exception will cause abnormal program termination. If we wish to prevent a function from throwing any exception, we may do so by making the type-list empty.

```
throw( ); //empty list
```

Example program:

```
#include<iostream.h>
#include<conio.h>
void test(int x) throw(int, double)
{
if (x==0) throw 'x';
//char else if(x==1)
throw x; //int
else if (x==-1) throw 1.0; //double
cout<<"End of function block";
}
void main()
{
try
{
cout << "Testing throw
restrictions";test(0);
test(1);
test(-1);
}
catch(char c)
{
cout<<"caught a character";
}
catch( int m)
{
cout<<"caught integer";
}
Catch(double d)
{
cout<<"caught a double";
}
Cout<<"End of main";
```

```
}
```

Output:

Testing rethrowing exception

Caught a character

End of main

Question Bank UNIT V**Multiple Choice Questions**

1. What consists in the family that is created by using a template?
A. Classes and functions B. Structures C. Variables D. Keywords
2. Which of the following can be handled by family of classes generated ,using a template
A. Different data types C. Same data types
B. Different variables D. Different identifiers
3. Which is the most significant feature that arises by using template classes?
A. Code readability **C. Code reusability**
B. Ease in coding D. Modularity in code
4. What can be passed by non-type template parameters during compile- time?
A. int C. float
B. constant expression D. char
5. What is a template?
A. A template is used to manipulate the class
B. A template is a formula for creating a generic class
C. A template is used for creating the attributes
D. A template is a data type
6. How many types of templates are there in C++?
A. 1 **B. 2** C. 3 D. 4
7. What is an exception?
A. Problem arising during compile time C. Problem in syntax
B. Problem arising during runtime D. Problem in IDE
8. What is a function template?
A. Creating a function without having to specify the exact type.
B. Creating a function with having an exact type.
C. Creating a variable with exact type.
D. None of the above.
9. Which are the two blocks that are used to check error and handle the error?
A. Try and catch C. Do and while
B. Trying and catching D. Try Do and Check
10. Which keyword can be used in template ?
A. Class **C. Both A and B**
B. typename D. Function
11. How many kinds of exceptions are present in C++

A. 4 **B. 2** C. 3 D. 1

12. Which of the below is not a keyword in exception handling mechanism?

A. Try **B. stop** C. catch D. Throw

1. What can a template class have?

- i) It can have More than one generic data type**
- ii) It can have Only one generic data type
- iii) It can have At most two data types
- iv) It can have Only generic type of integers and not characters

2. How to declare a template?

- i) template ()
- ii) template{ }
- iii) template < >**
- iv) template[]

3. If template class is defined, is it necessary to use different types of data for each call?

- i) No, not necessary**
- ii) No, but at least two types must be there
- iii) Yes, to make proper use of template
- iv) Yes, for code efficiency

4. What may be the name of the parameter that the template should take?

- i) same as variable
- ii) same as class
- iii) same as function
- iv) same as template

5. Which is the statement used to catch all the exceptions.

- i) catch{...}
- ii) catch[...]
- iii) catch(...)**
- iv) catch(void x)

6. Which is the exception type for Error such as “out-of-range index”.

- i) **Synchronous** B. Asynchronous C. type-safe D. functional

7. Which block must be immediate after the try block without any code between them.

- i) Throw B. Stop **C. Catch** D. type

8. Choose which is the correct syntax of throw statement from below

- i) Throw()** B. throw[exception] C. throw { }
- D. throw(exception)**

9. Which block can have the code of statements which may cause abnormal termination of the program

- i) Try**
- ii) Catch
- iii) Finally
- iv) Throw

10. Which of the following statements are true?

1. Catch should always be placed last in the list of exception handlers.

2. C++ compiler does not check if any exception is caught or not.

A. Only 1 **B. both 1 and 2** C. Only 2 D. Both are wrong

23. What is the validity of template parameters?

A. **inside that block only** C. whole program

B. inside the class D. any of the mentioned

24. Why do we need to handle exceptions?

A. To avoid syntax errors **C. To prevent abnormal termination of program**

B. To avoid semantic errors D. To save memory

25. Can you predict when can an exception arise?

A. When Input is fixed **C. When Input given is invalid**

B. When Input is some constant value of program D. When Input is valid

Long Answer questions

1. Plan on how to declare the class templates? Explain with syntax and example.

2. Plan on how to define the function templates? Explain with syntax and example.

3. Construct a program and illustrate how to throw an exception? Explain

4. Experiment with the use multiple catch statements in a program? Explain.

5. Construct a program to demonstrate how to re-throw an exception? Explain

6. Discuss the Importance of templates in programming Explain with the help of an example

7. Elaborate on basics of exception handling having two kinds of exceptions.

8. Elaborate about Class templates with multiple parameters.

9. Analyze overloading of template functions? Explain with example.

10. Elaborate on Exception Handling mechanism with example.

11. Analyze about specified exception in detail

